

Nástroj pro distribuované zpracování dat

Tool for Distributed Processing of Data

Filip Rajdus

Bakalářská práce

Vedoucí práce: Ing. Marek Běhálek, Ph.D.

Ostrava, 2025

Zadání bakalářské práce

Student:

Filip Rajdus

Studijní program:

B0613A140014 Informatika

Téma:

Nástroj pro distribuované zpracování dat
Tool for Distributed Processing of Data

Jazyk vypracování:

čeština

Zásady pro vypracování:

Hlavním cíle práce bude navrhnout nástroj, který usnadní naplánování distribuovaného výpočtu na serveru. Motivací pro zadání práce je zpracování dat z optického mapování. Aktuálně je na školním serveru, určeném pro tento úkol, několik desítek vzorků, což představuje desítky TB dat a stovky tisíc souborů. Nástroj by měl dvě části. První, jakýsi konfigurátor, by umožnil vyhledat na serveru (například podle přípony) skupiny souborů, pro tyto soubory by pak částečně automatizovaně sestavit skripty, které by specifikovaly nějaký výpočet. Druhá část nástroje by pak tyto skripty na serveru vykonala. Nástroj by informoval o průběhu výpočtu a jeho dokončení. Hlavním přínosem nástroje by byla jakási automatizace různých nastavení, například cest k souborům.

Cíle práce lze shrnout v těchto bodech:

1. Seznamte se s problémy při provádění výpočtů nad daty z optického mapování.
2. Navrhněte nástroj, který umožní plánování těchto výpočtů poloautomatickým způsobem.
3. Navržené řešení implementujte.
4. Rozšiřte řešení i různé doplňkové činnosti, jako je přehled o průběhu výpočtu či automatické sbírání výsledků.

Seznam doporučené odborné literatury:

- [1] Bionano Genomics: Saphyr Whole Genome Imaging Instrument, dostupné online: <https://bionanogenomics.com/products/saphyr/>
- [2] M. van Steen and A.S. Tanenbaum, Distributed Systems, 3rd ed., distributed-systems.net, 2017.

Formální náležitosti a rozsah bakalářské práce stanoví pokyny pro vypracování zveřejněné na webových stránkách fakulty.

Vedoucí bakalářské práce: **Ing. Marek Běhálek, Ph.D.**

Datum zadání: 01.09.2023

Datum odevzdání: 15.07.2025

Garant studijního programu: doc. Mgr. Miloš Kudělka, Ph.D.

V IS EDISON zadáno: 03.06.2025 16:43:22

Abstrakt

Cílem této bakalářské práce je vytvořit nástroj pro plánování a správu distribuovaných výpočtů na serveru určeném pro zpracování dat z optického mapování. Server obsahuje desítky terabajtů dat v podobě stovek tisíc souborů, kde ruční zpracování představuje výraznou časovou zátěž.

Hlavní přínos práce spočívá v nástroji rozděleném do dvou částí. První částí je konfigurátor, který umožňuje uživateli organizovat vstupní data pomocí filtrů, přiřazovat je k existujícím výpočetním skriptům a nastavovat parametry paralelního zpracování. Druhou částí je vykonávací modul, který zajišťuje plánování úloh na serverových zdrojích, správu paralelních výpočtů a monitoring průběhu zpracování.

Práce zahrnuje analýzu specifických požadavků na zpracování dat z optického mapování a představuje řešení, které umožňuje efektivní práci s velkým množstvím souborů.

Klíčová slova

distribuované zpracování dat; optické mapování; bakalářská práce

Abstract

The goal of this bachelor thesis is to create a tool for planning and managing distributed calculations on a server used for processing optical mapping data. The server contains terabytes of data in the form of hundreds of thousands of files, where manual processing is very time-consuming.

The main benefit of this work is a tool divided into two parts. The first part, the configurator, helps the user organize input data using filters, assign them to existing calculation scripts, and set parameters for parallel processing. The second part, the execution module, provides task scheduling on server resources, manages parallel calculations, and monitors the processing progress.

This thesis also analyzes the specific requirements for processing optical mapping data and presents a solution that allows efficient work with large numbers of files.

Keywords

distributed data processing; optical mapping; bachelor's thesis

Poděkování

Rád bych poděkoval svému vedoucímu práce Ing. Markovi Běhálkovi, Ph.D., za jeho přátelský přístup, cenné rady a konstruktivní kritiku, které významně přispěly ke kvalitnějšímu výsledku této práce. Děkuji také své rodině a blízkým za jejich podporu během studia.

Obsah

Seznam použitých symbolů a zkratek	6
Seznam obrázků	7
Seznam tabulek	8
Seznam výpisů zdrojového kódu	9
1 Úvod	10
2 Optické mapování	11
2.1 Co je optické mapování	11
2.2 Charakteristika dat z optického mapování	12
2.3 Datové typy v optickém mapování	12
2.4 Výzvy při zpracování dat z optického mapování	13
2.5 Existující nástroje	14
3 Analýza problému	16
3.1 Množství dat	16
3.2 Organizace souborů	17
3.3 Správa dat a výsledků	17
3.4 Návrh řešení	19
4 Vývoj nástroje	23
4.1 Funkcionalita a architektura nástroje	23
4.2 Implementace	27
4.3 Scénáře	35
4.4 Testování	42
5 Závěr	49
Literatura	50

Seznam použitých zkratek a symbolů

DNA	– Deoxyribonukleová kyselina, Nositel genetické informace organismů
JSON	– JavaScript Object Notation
SQL	– Structured Query Language
API	– Application Programming Interface

Seznam obrázků

2.1	Proces mapování DNA. [15]	12
3.1	Komponentní diagram - Návrh architektury programu	20
3.2	Schéma klíčových kroků v uživatelském scénáři	22
4.1	Původní struktura v textovém souboru	33
4.2	Struktura BNX dat po načtení - hlavička (vlevo) a data (vpravo)	33
4.3	Activity diagram vyhledání souborů pro analýzu	37
4.4	Sekvenční diagram scénáře vyhledávání v souborech	39
4.5	Activity diagram paralelního zpracování	41
4.6	Použití funkce filter()	43
4.7	Paralelní spuštění skriptu na 20 procesorových jádrech	45
4.8	Porovnání času zpracování v závislosti na velikosti souboru	47

Seznam tabulek

2.1	Porovnání Apache Hadoop a Apache Spark	15
4.1	Porovnání rychlosti výběru souborů	44
4.2	Porovnání rychlosti načítání souborů při různých konfiguracích	46
4.3	Časy zpracování podle velikosti souboru	47

Seznam výpisů zdrojového kódu

4.1	Inicializace vyhledávání: načtení všech souborů a filtrace pouze adresářů	29
4.2	Rekurzivní prohledávání složek, pokud je aktivní	30
4.3	Detekce duplicit s porovnáním metadat	30
4.4	Identifikace typů hlaviček	31
4.5	Konverze hodnot podle hlavičky	31
4.6	Transformace do slovníku	32
4.7	Příklad práce s načtenými daty	32
4.8	Rozdělení vstupu do chunků podle počtu jader	34
4.9	Inicializace sdílených dat ve workerech	34
4.10	Základ paralelního zpracování pomocí funkce pfor()	35

Kapitola 1

Úvod

S rostoucím množstvím dat v oblasti vědy a výzkumu se stává klíčovým úkolem jejich efektivní zpracování. Jednou z oblastí, která čelí těmto výzvám, je zpracování dat z optického mapování. Tato data představují obrovské objemy informací, často v řádech desítek terabajtů, rozdělených do stovek tisíc souborů. Zpracování těchto dat na serverech vyžaduje plánování výpočtů, správu velkého množství souborů a efektivní provádění výpočetních operací.

Pokud by byla realizace těchto výpočtů prováděna manuálně, jednalo by se o časově náročný proces, který by byl náchylný k chybám, například při specifikaci cest k souborům nebo při nastavování výpočetních skriptů. Automatizace a vytvoření poloautomatických nástrojů by tyto procesy nejen usnadnily a urychlily, ale také by minimalizovaly riziko chyb.

Cílem této práce je navrhnout a implementovat nástroj, který usnadní a urychlí zpracování dat z optického mapování. Nástroj umožní vyhledání souborů podle specifických kritérií, poloautomatickou tvorbu skriptů pro specifikaci výpočtů a jejich následné spouštění na serveru. Díky tomu pomůže efektivně řídit velké množství úloh a minimalizovat chybovost při práci s daty. V dalších kapitolách bude představena teorie optického mapování, analýza problému spolu s návrhem řešení a rozbor vytvořeného nástroje, včetně implementace a ukázkových scénářů použití.

Kapitola 2

Optické mapování

„Poznání struktury a funkce dědičné informace je klíčem k porozumění podstaty veškerého života. Díky moderním technologiím sekvenování DNA roste počet organismů, u nichž byla dědičná informace přečtena. Sestavené genomové¹ sekvence mají však obvykle daleko k dokonalosti. Optické mapování představuje nástroj, který může přispět k jejich vylepšení, ale své uplatnění nalézá také na poli studia strukturní variability mezi genomy.“ [15]

Moderní metody optického mapování generují rozsáhlé soubory experimentálních dat, které vyžadují specializované přístupy ke zpracování. Tato kapitola se věnuje technickým aspektům technologie, zejména struktuře a charakteristikám výsledných datových souborů ve formátech BNX, CMAP a XMAP.

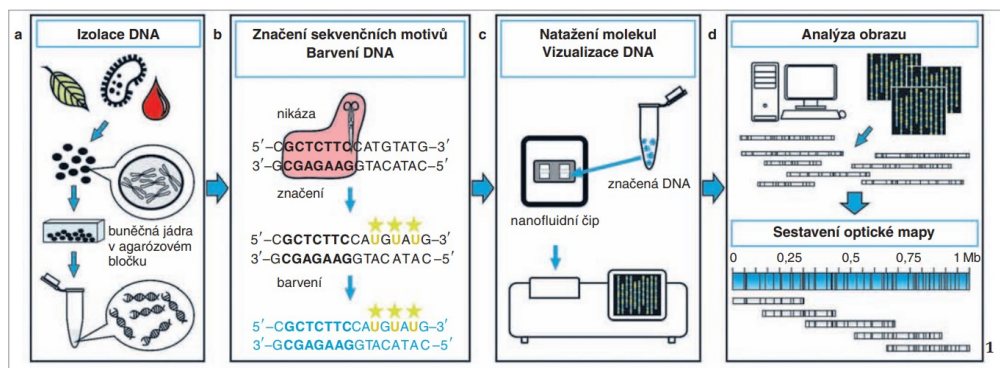
2.1 Co je optické mapování

Optické mapování je technologie využívaná k analýze, rozboru a vizualizaci struktury DNA nebo jiných biologických molekul. Tyto molekuly hrají klíčovou roli jako nositelé genetické informace u všech živých organismů i některých virů. DNA obsahuje instrukce nezbytné pro vývoj, růst a fungování organismů. [5, 15]

Jak znázorňuje obrázek 2.1, proces mapování DNA začíná izolací molekul, které jsou následně lineárně uspořádány a analyzovány pomocí fluorescenčních značek. Tento vizualizační přístup umožňuje identifikovat strukturní variace, které by při tradiční analýze zůstaly neodhalené. [15]

Princip optického mapování spočívá v získání dlouhých úseků DNA, jejich následném zpracování a zobrazení pomocí optického mikroskopu nebo jiných pokročilých zobrazovacích technologií. Výsledkem jsou charakteristické „mapy“, které zachycují specifické vzory na základě přítomnosti určitých sekvencí nebo enzymatických štěpných míst. Tyto mapy se následně analyzují a porovnávají s referenčními daty, což umožňuje identifikovat genetické variace, mutace nebo další vlastnosti, například strukturální změny spojené s onemocněními. [5, 15]

¹Genom je kompletní sada genetické informace organismu.



Obrázek 2.1: Proces mapování DNA. [15]

2.2 Charakteristika dat z optického mapování

Data získaná metodou optického mapování mají několik výrazných charakteristik, které významně ovlivňují jejich uchovávání, zpracování a další analýzu. Vyznačují se především velmi velkým objemem, jednotlivá měření mohou generovat desítky až stovky gigabajtů dat. Takto rozsáhlé datasety kladou vysoké nároky jak na úložiště, tak i na jejich následné zpracování.

Další významnou vlastností je rozmanitost datových formátů, ve kterých jsou výsledky ukládány. Mezi nejčastější patří surová data, soubory obsahující výsledky preprocessingu (například detekce značek nebo segmentace) a také různé typy anotací či metadat, které popisují experimentální podmínky nebo parametry analýzy. Tato různorodost zvyšuje složitost následného zpracování a vyžaduje specializované nástroje pro jejich správnou interpretaci.

Charakteristické je také to, že výsledná data bývají rozdělena do velmi velkého počtu souborů, běžně jde o stovky či tisíce jednotlivých souborů. Toto členění výrazně ztěžuje manuální správu a kontrolu celého datasetu.

V neposlední řadě je nutné zmínit hierarchickou strukturu úložiště, v němž jsou tato data uložena. Adresářová struktura bývá složitá, často obsahuje několik úrovní vnořených složek, které odrážejí například experimentální sady, fáze zpracování nebo specifické části analyzovaných vzorků. I když taková hierarchie umožňuje tematické třídění a logické uspořádání dat, zároveň podstatně komplikuje jejich vyhledávání, přenos nebo automatizované zpracování.

Celkově tedy práce s daty z optického mapování vyžaduje robustní softwarové nástroje, schopné efektivně pracovat s velkými objemy souborů, zvládat různé datové formáty a podporovat operace v prostředí složité adresářové struktury.

2.3 Datové typy v optickém mapování

Optické mapování Bionano technologie využívá hierarchii specializovaných datových formátů .BNX, .CMAP a .XMAP, které systematicky zachycují celý proces od surových molekulárních dat až po finální

porovnání. Tyto formáty tvoří základní datovou infrastrukturu pro moderní analýzy, umožňující jak sestavování genomů, tak detekci rozsáhlých strukturních variant. Tyto formáty se vzájemně doplňují a společně umožňují komplexní analýzu genomu. [11]

Vyjmenované formáty sdílejí společnou strukturu, která usnadňuje jejich čtení a zpracování. Každý soubor se skládá ze tří základních částí. První je hlavička obsahující metadata, jako jsou informace o verzi formátu nebo parametrech analýzy, přičemž tyto řádky jsou obvykle označeny znakem #. Druhou částí je popis tabulky označený #h nebo #f, který definuje názvy sloupců a datové typy jednotlivých polí a slouží jako schéma dat. Nakonec soubor obsahuje datovou část, která bývá nejrozsáhlejší a obsahuje samotná měřená data či výsledky analýzy ve strukturované formě podle definovaného schématu. Tyto soubory mají desítky tisíc až miliony řádků v závislosti na množství změřených molekul. Tato struktura zajišťuje přehlednost všech tří formátů a umožňuje efektivní průchod dat při analýze.

2.3.1 Formát .BNX

Formát .BNX je primární výstup přístrojů Bionano. Obsahuje surová data jednotlivých molekul DNA, například jejich délku, pozice fluorescenčních značek, intenzity signálu a poměr signálu k šumu (SNR). Používají se jako základ pro kontrolu kvality vstupních dat - rozdělují molekuly na spolehlivě rozpoznané (s vysokým Q-score) a nedostatečně přesné. Tyto soubory slouží jako vstupní data pro sestavování genomových map. [9, 11]

2.3.2 Formát .CMAP

Po sestavení vzniká formát .CMAP, který obsahuje sjednocené pozice značek více molekul. Na rozdíl od .BNX už neukládá surové intenzity, ale pouze výsledné pozice značek ve složených úsecích. Tento formát se používá jako referenční mapa pro další analýzy. [7, 11]

2.3.3 Formát .XMAP

Formát .XMAP popisuje výsledky zarovnání dvou .CMAP souborů. Určuje vzájemnou korespondenci mezi úseky DNA, včetně jejich orientace, pozic mezer a skóre spolehlivosti zarovnání. Díky tomu je zásadní pro detekci strukturních variant v genomu. [8, 11]

2.4 Výzvy při zpracování dat z optického mapování

Zpracování dat z optického mapování představuje řadu specifických výzev, které je nutné při návrhu nástroje zohlednit. Mezi klíčové požadavky patří především přesné vyhledávání relevantních souborů, například prostřednictvím selekce podle přípon, velikosti nebo typu dat. Dále je důležité zajistit efektivní správu cest k jednotlivým souborům, kdy automatizace při specifikaci cest výrazně snižuje riziko chyb a zároveň šetří čas při přípravě zpracování.

Další výzvou je plánování výpočtů, které často vyžaduje rozdělení rozsáhlých úloh do menších částí, které lze zpracovávat paralelně s cílem urychlit celkový průběh. Je třeba zmínit nutnost optimalizace zdrojů, tedy efektivní využití dostupné výpočetní kapacity serveru, což umožňuje minimalizovat dobu trvání jednotlivých výpočtů a maximalizovat propustnost celého systému.

Významnou roli hraje také schopnost porozumět vnitřní struktuře dat a orientovat se v často složitých a hierarchicky uspořádaných adresářových strukturách. Bez důkladného pochopení této organizace je následná analýza, návrh nebo správa souborů značně obtížná a časově náročná.

2.5 Existující nástroje

Tato kapitola představuje existující frameworky pro distribuované zpracování dat (Apache Hadoop, Apache Spark a Nextflow), které slouží jako referenční modely pro návrh specializovaného řešení. Zatímco tyto nástroje poskytují univerzální řešení, navrhovaný systém se zaměřuje výhradně na specifické potřeby optického mapování, konkrétně na práci s formáty BNX, CMAP a XMAP.

2.5.1 Apache Hadoop

Apache Hadoop je framework pro distribuované zpracování velkých objemů dat na clusteru počítačů. Využívá jednoduchý model programování a poskytuje škálovatelné a odolné řešení. Hlavním cílem je zjednodušit použití sítě mnoha počítačů k řešení problémů, které zahrnují významné objemy dat a výpočtů. Nabízí softwarový framework pro distribuované ukládání a zpracování za pomoci programovacího modelu MapReduce. Všechny moduly jsou postaveny s předpokladem, že dochází k hardwarovým výpadkům a framework by tak měl být schopen tyto výpadky automaticky adresovat. [2]

2.5.2 Apache Spark

Apache Spark je open-source distribuovaný systém pro zpracování velkých objemů dat. Využívá ukládání do mezipaměti a optimalizované provádění dotazů k umožnění rychlých analytických dotazů na data jakékoli velikosti. Nabízí vývojová rozhraní API v jazycích Java, Scala, Python a R, která umožňují opětovné použití kódu v různých pracovních zátěžích, například dávkové zpracování, interaktivní dotazy, analýzy v reálném čase, strojové učení a zpracování grafů. Spark je široce používán napříč průmyslovými odvětvími, včetně společností FINRA, Yelp, Zillow, DataXu, Urban Institute a CrowdStrike. [16]

Jak ukazuje tabulka 2.1, zásadní rozdíl mezi Apache Hadoopem a Apache Sparkem spočívá v přístupu ke zpracování dat. Hadoop provádí operace převážně na disku, zatímco Spark využívá zpracování přímo v paměti, čímž poskytuje značné zrychlení. Výběr technologie záleží hlavně na konkrétních požadavcích.

Tabulka 2.1: Porovnání Apache Hadoop a Apache Spark

	Apache Hadoop	Apache Spark
Architektura	Dávkové zpracování založené na disku	Zpracování v paměti s možností ukládání na disk
Model zpracování	Pouze dávkové zpracování (MapReduce)	Dávkové, interaktivní, streamování a iterativní zpracování
Výkon	Pomalejší kvůli diskovým operacím	Až 100× rychlejší pro výpočty v paměti
Odolnost proti chybám	Zajištěna replikací dat (obvykle 3×)	Zajištěna pomocí RDD lineage a checkpointingu
Uživatelská přívětivost	Nížší úroveň API, vyžaduje více kódu	Vysoká úroveň API (Scala, Python, Java, R) a bohaté knihovny
Latence	Vysoká latence (dávkové zpracování)	Nízká latence (zpracování v paměti)
Zpracování dat	Primárně dávkové zpracování	Dávkové, streamování v reálném čase, strojové učení, grafové analýzy
Využití paměti	Disk-based (minimální nároky na RAM)	Náročné na paměť (vyžaduje více RAM)
Náklady	Nížší hardwarové nároky	Vyšší hardwarové náklady (potřeba více RAM)
Ekosystém	HDFS, YARN, MapReduce, HBase, atd.	Spark Core, Spark SQL, Spark Streaming, MLlib, GraphX

[1, 2, 16]

2.5.3 Nextflow

Nextflow je open-source nástroj pro vytváření a správu vědeckých výpočetních workflow, který je obzvláště populární v bioinformatice a dalších oblastech pracujících s rozsáhlými datovými sadami. Nextflow nabízí integrovanou podporu kontejnerů, konkrétně pro Docker, což zajišťuje reprodukovatelnost výpočtů. Dále využívá reaktivní architekturu, která umožňuje automatickou paralelizaci úloh na základě dostupnosti dat. Kromě toho poskytuje pokročilé možnosti monitorování průběhu výpočtů a jejich obnovy v případě potřeby. [4, 12]

Přestože nástroje jako Hadoop, Spark či Nextflow poskytují univerzální řešení pro distribuované zpracování dat, navrhované řešení se zaměřuje na specifické potřeby zpracování dat z optického mapování souborů BNX, CMAP a XMAP. Cílem je vytvořit nástroj, který usnadní práci s velkým množstvím souborů a umožní spouštění skriptů pro výpočty v prostředí serveru.

Zmíněné nástroje v tomto kontextu slouží spíše jako příklad softwarového návrhu a inspirace pro vývoj vlastního řešení. Navrhovaný nástroj tedy není konkurencí těmto frameworkům, ale spíše experimentem zaměřeným na práci s molekulárními daty.

Kapitola 3

Analýza problému

Tato kapitola provádí analýzu technických výzev spojených se zpracováním dat optického mapování. Na základě identifikovaných problémů vytváří konkrétní požadavky na softwarové řešení a prezentuje návrh nástroje. Závěr kapitoly pak demonstruje praktickou aplikaci navrhovaného řešení na typické úlohy.

3.1 Množství dat

Hlavním problémem při práci s daty z optického mapování je jejich velký objem, jehož zpracování může být časově i výpočetně velmi náročné. Jedním z účinných způsobů, jak tento problém řešit, je využití filtračních technik, které umožňují pracovat pouze s relevantní podmnožinou dat. U menších datasetů lze použít jednoduchý přístup, kdy jsou soubory nebo složky explicitně určeny předem. Pokud však množství dat narůstá, je nezbytné použít pokročilejší metody založené na kombinaci různých kritérií, například názvu nebo přípony souboru, velikosti, data poslední úpravy nebo i obsahu samotného souboru. Tato kritéria lze dále kombinovat a vytvářet tak složitější výběrová pravidla s využitím množinových operací, jako je sjednocení, průnik nebo rozdíl.

Pro urychlení práce je vhodné využít indexaci, která výrazně zkracuje dobu přístupu k požadovaným souborům. U rozsáhlých dat se nabízí využít paralelní zpracování, kdy jsou jednotlivé části dat analyzovány současně napříč více vlákny, jádry nebo dokonce servery.

Při návrhu a implementaci těchto filtrů je však nutné řešit několik praktických problémů. Mezi časté výzvy patří volba vhodných kritérií: příliš obecná mohou zahrnout nežádoucí soubory, naopak příliš přísná mohou vyloučit i potřebná data. Dále je třeba ošetřit situace spojené s omezeným přístupem nebo ochranou souborů, kde mohou být vyžadovány speciální přístupové mechanismy. Velkou roli hraje i dostatečné logování a dokumentace všech provedených operací, které usnadňují zpětnou kontrolu a odhalení případných chyb.

Dalším specifickým problémem bývá nutnost upravovat nebo obnovovat dříve definované výběry. Pokud není k dispozici možnost návratu k původním nastavením, může dojít ke ztrátě informací

a zkomplikování následných analýz. Proto je vhodné zajistit zálohování výběrových pravidel nebo zavést historii filtrů, aby bylo možné se vracet k předchozím pravidlům nebo výsledkům.

3.2 Organizace souborů

Dalším problémem při práci s daty je jejich samotná struktura a způsob uložení. V systémech, kde jsou soubory různých typů a formátů rozdělené do mnoha složek, může být složité a časově náročné identifikovat soubory relevantní pro zpracování. Tento problém je často umocněn tím, že požadované soubory mohou být rozmístěny ve více úrovních složek.

Pro řešení této situace lze využít rekurzivní filtrování, které umožňuje prohledávat soubory včetně všech podsložek. Rekurzivní přístup však přináší i specifické problémy, například cyklické odkazy nebo příliš velkou hloubku složek. To může vést k neefektivnímu zpracování dat nebo až k selhání procesu, pokud tyto situace nejsou správně ošetřeny. Dalším omezením může být maximální hloubka rekurze nastavená v systému. Pokud je tento limit překročen, může dojít k přerušení vyhledávání.

3.3 Správa dat a výsledků

Efektivní správa dat a výsledků je klíčovou součástí distribuovaného zpracování. Správná organizace a ukládání dat zajišťuje nejen rychlý přístup, ale také možnost následného využití pro analýzu nebo další zpracování. Jedním z hlavních cílů při správě dat je jejich strukturování tak, aby bylo možné snadno vyhledávat relevantní soubory a výsledky. Takto uspořádaná data umožňují rychlý a efektivní přístup k potřebným informacím bez zbytečného hledání.

3.3.1 Výsledky

Pro ukládání výsledků je nutné zvážit následující aspekty, jako je struktura dat, formát nebo třeba i komprese. Výsledky by měly být uloženy v přehledné a efektivní struktuře, která umožňuje rychlé vyhledávání a snadnou manipulaci. Komprese může být použita pro optimalizaci využití úložiště, zejména v případě velkých objemů dat. Výsledky zpracování by měly být snadno dostupné pro další použití. Toto umožňuje rychlé a efektivní prohledávání a práci s daty v závislosti na jejich potřebách.

3.3.2 Metadata

Kromě samotných dat je důležité ukládat i metadata, která poskytují kontext a usnadňují práci s výsledky. Metadata mohou zahrnovat použité nastavení při spouštění filtrů, parametry pro soubory, původ dat, informace o čase zpracování a další pokročilé funkce, které jsou právě použity. Tato metadata jsou nezbytná pro zpětnou kontrolu a zajištění konzistence mezi různými spuštěními procesu.

3.3.3 Zálohování a obnova

Zálohování a obnova, včetně historie spuštěných příkazů, představují klíčové nástroje, které umožňují uživateli vrátit se k předchozím krokům v případě, že není spokojen s aktuálním výsledkem nebo stavem. Tato historie usnadňuje sledování změn a experimentování s různými konfiguracemi bez rizika ztráty předchozích verzí výsledků.

3.3.4 Formáty pro uložení dat a výsledků

Data a výsledky z distribuovaného zpracování mohou být uloženy v různých formátech v závislosti na požadavcích projektu, specifikách dat a účelu zpracování. Mezi nejběžnější formáty patří:

3.3.4.1 CSV (Comma Separated Values)

Často používaný pro tabulková data, jednoduchý formát pro uložení dat v podobě textových souborů, který je snadno čitelný a přenosný mezi různými programy.

3.3.4.2 JSON (JavaScript Object Notation)

Používá se pro strukturovaná data, jako jsou objekty a pole. Je čitelný pro člověka a široce používaný v webových aplikacích a při výměně dat mezi systémy.

3.3.4.3 XML (Extensible Markup Language)

Formát pro strukturovaná data, který se používá pro přenos informací mezi systémy. Je flexibilní a lze ho použít pro složitější datové struktury.

3.3.4.4 SQL (Structured Query Language)

Strukturované formáty, kde data jsou uložena v relačních databázích. Často používané pro organizování velkých objemů strukturovaných dat s možností vysoce efektivního dotazování a manipulace.

3.3.4.5 Logovací soubory

Logovací soubory jsou textové soubory, které primárně neslouží k ukládání výsledků, ale uchovávají historii provedených operací systému či aplikace. Tímto způsobem úzce souvisejí s ukládáním dat a výsledků. Slouží k informování o průběhu zpracování, monitorování procesů, diagnostice chyb a analýze chování během provozu.

3.4 Návrh řešení

Tato sekce představuje klíčové aspekty návrhu řešení pro efektivní zpracování dat. Nejprve jsou definovány požadavky, které systém musí splňovat, včetně pokročilých možností manipulace s výběry souborů, optimalizovaného paralelního zpracování a správy příkazů s notifikacemi. Dále je popsána navržená architektura nástroje, která je rozdělena do modulů zajišťujících správu příkazů, selekci a načítání dat, zpracování, historii operací a komunikaci s uživatelem. Tato struktura umožňuje získat přehled o fungování programu a představit si, jak jednotlivé části spolupracují při zpracování rozsáhlých dat. Nakonec je uveden příklad uživatelského scénáře, který demonstruje praktické použití systému při vyhledávání, filtrování a paralelním zpracování dat s následným exportem výsledků a zasláním reportu uživateli.

3.4.1 Požadavky

Následující požadavky vymezují klíčové funkce a vlastnosti, které by měl navržený systém splňovat, aby odpovídal zadání a vhodně pracoval s daty.

3.4.1.1 Pokročilé vyhledávání na serveru

Systém musí umožnit vyhledávání v zadaném adresáři i rekurzivně v celé adresářové struktuře. Vyhledávání by mělo podporovat použití zástupných znaků (např. `*.txt`, `data__.csv`) a regulárních výrazů. Dále by mělo být možné provádět fulltextové vyhledávání v obsahu souborů, detekovat a spravovat duplicitní soubory a filtrovat výsledky podle názvu, přípony nebo velikosti souboru.

3.4.1.2 Manipulace s výběrem

Pro složitější tvoření výběru vstupních souborů by měl systém umožnit provádět množinové operace s výběry, jako jsou sjednocení, průnik a rozdíl. Výběry by mělo být možné ukládat do pojmenovaných skupin a exportovat do formátů CSV nebo JSON. Součástí požadavků je také možnost práce s historií vyhledávání, umožňující návrat k předchozím výsledkům.

3.4.1.3 Optimalizované zpracování

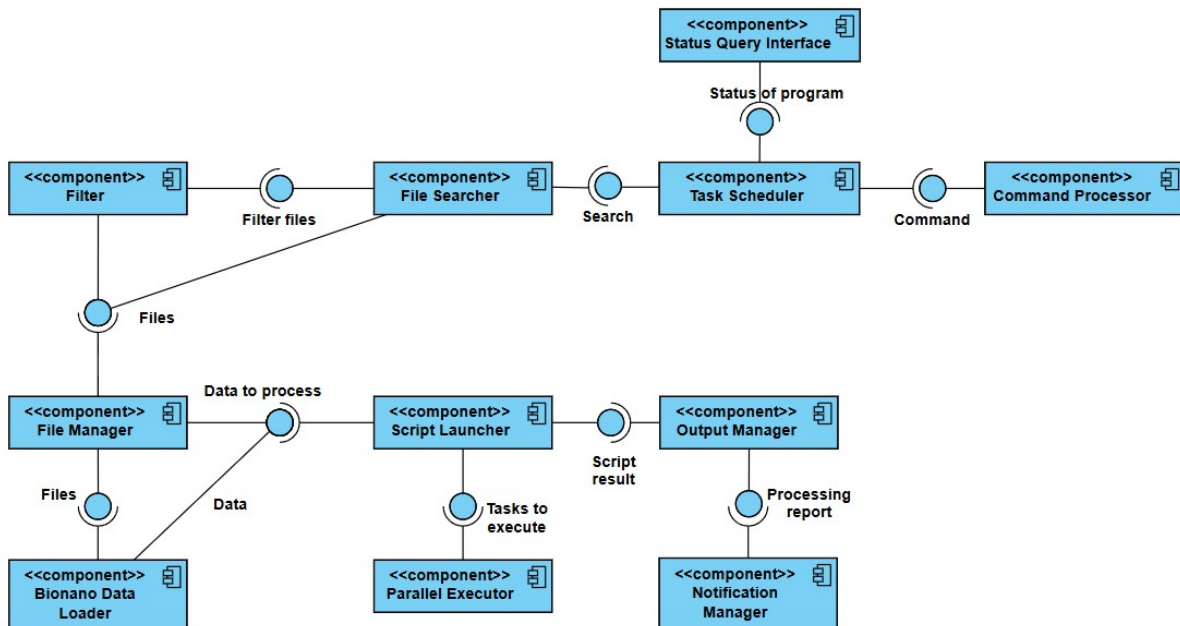
Vzhledem k většímu množství souborů by systém měl podporovat efektivní zpracování dat prostřednictvím paralelního spouštění výpočtů a převodu Bionano souborů (formáty `.bnx`, `.cmap`, `.xmap`) do strukturovaného formátu JSON. Paralelní zpracování umožňuje současné provádění více úloh, což výrazně zkracuje celkový čas potřebný k analýze rozsáhlých datových sad. Převod do formátu JSON pak zajišťuje snadný a flexibilní přístup k datům pro další analýzy.

3.4.1.4 Zpracování příkazů a notifikace

Zásobník příkazů umožní sekvenční zpracování s automatickým spouštěním v zadaném pořadí. Po dokončení bude odeslána e-mailová notifikace obsahující informace o čase dokončení a seznam provedených příkazů.

3.4.2 Návrh architektury

Celkovou architekturu navrženého nástroje pro plánování distribuovaného výpočtu znázorňuje obrázek 3.1. Architektura je rozdělena do několika hlavních modulů, které spolu komunikují. Hlavními komponentami jsou Status Query Interface, tedy vlákno pro příjem a obsluhu příkazů od uživatele, modul pro správu vstupních souborů a jejich selekci na základě zadaných filtrů a Bionano Data Loader, který zajišťuje načtení a přípravu dat pro zpracování. Výsledky zpracování jsou poté předány modulu pro správu historie, kde se uchovávají záznamy o provedených operacích a výsledcích. Součástí architektury je také modul pro notifikace, který umožňuje informovat uživatele (například e-mailem s výstupní zprávou) o dokončení úloh. Tyto komponenty si umožňují předávat informace a postupovat pipeline až k samotnému provedení výpočtu.



Obrázek 3.1: Komponentní diagram - Návrh architektury programu

3.4.3 Příklad uživatelského scénáře

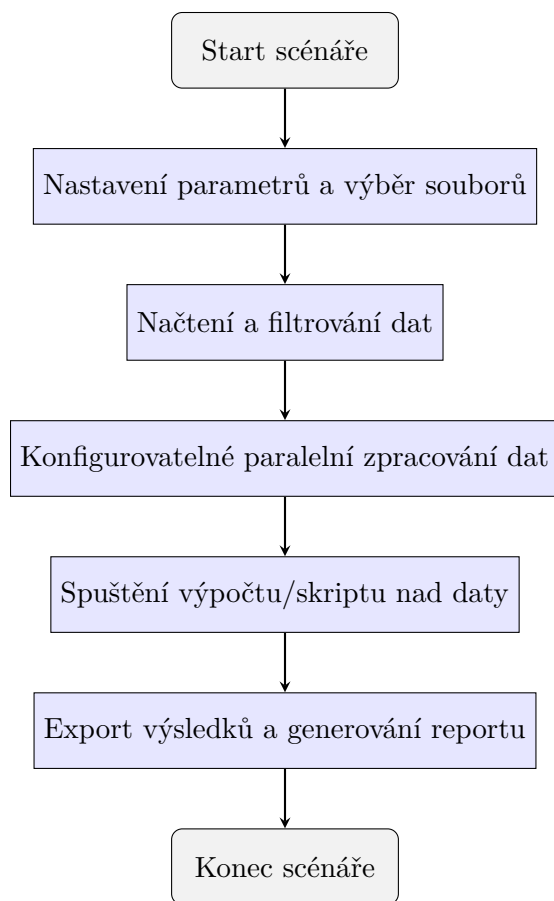
Uživatel začíná tím, že určí konkrétní adresář, ve kterém chce vyhledávat soubory pomocí funkce `filter()`, případně explicitně definuje soubory. Lze prohledávat pouze aktuální složku, nebo re-

kurzivně procházet celou adresářovou strukturu a vybrat všechny soubory. Zároveň může aplikovat filtry, které omezí rozsah dat pouze na relevantní soubory, například s příponou `.bnx`. Následně jsou tyto soubory filtrovány podle zvolených kritérií, jako je datum poslední úpravy či velikost, čímž se zajistí, že do dalšího zpracování vstoupí pouze aktuální a důležitá data. Vybrané bionano soubory jsou poté načteny do paměti aplikace pomocí funkce `load()`, která zajistí jejich přečtení a zpracování.

Každý krok celého procesu je konfigurovatelný, takže uživatel může snadno měnit filtrovací pravidla, nastavovat různé parametry načítání nebo přidávat vlastní scénáře zpracování podle aktuálních potřeb. Tyto scénáře lze ukládat a znovu využívat. Zároveň lze výběr souborů libovolně kombinovat, aby bylo možné najít ideální soubory pro konkrétní účel.

V dalším kroku uživatel definuje konkrétní podmínky pro analýzu dat, například filtruje molekuly podle délky, kvality signálu nebo směru skenu. Tato analýza může být následně spuštěna paralelně, s možností nastavení počtu procesorových jader, čímž se významně zkracuje doba zpracování. Po dokončení analýzy jsou výsledky exportovány do požadovaného formátu. Konečně může být automaticky vygenerován a odeslán report, který shrnuje klíčové informace o průběhu a výsledcích analýzy, což usnadňuje jejich další vyhodnocení a dokumentaci.

Na obrázku 3.2 je znázorněn postup uživatelského scénáře, který začíná nastavením parametrů a výběrem souborů, pokračuje načtením a filtrováním dat, dále zahrnuje paralelní konfigurovatelné zpracování a spuštění výpočtu nebo skriptu nad daty, a nakonec končí exportem výsledků a generováním reportu. Diagram zobrazuje jednotlivé fáze a jejich posloupnost.



Obrázek 3.2: Schéma klíčových kroků v uživatelském scénáři

Kapitola 4

Vývoj nástroje

Navrhovaný nástroj řeší tři klíčové aspekty práce s molekulárními daty optického mapování: efektivní vyhledávání a filtrování souborů ve formátech BNX, CMAP a XMAP, jejich přípravu pro následnou analýzu a optimalizované rozdělení výpočetních úloh pro paralelní zpracování. Tento přístup umožňuje zefektivnit práci s velkými datovými soubory a zároveň zajistit rychlou a podrobnou analýzu. Pomocí rozdělení systému je možné jednotlivé části nezávisle upravovat a vylepšovat podle konkrétních potřeb uživatele. Následující část detailně popisuje architekturu tohoto systému, přičemž se zaměřuje na strukturu jednotlivých komponent, implementaci klíčových algoritmů, propojení mezi vyhledávacími a výpočetními funkcemi.

4.1 Funkcionalita a architektura nástroje

Tato část se zaměřuje na detailní popis jednotlivých komponent a jejich vzájemných vazeb. Představuje hlavní funkce programu, způsoby jejich realizace a principy architektury. Díky tomuto přehledu je možné lépe porozumět organizaci systému, způsobu práce s daty a možnostem rozšiřování. Podsektce dále rozebírají konkrétní funkční moduly, technické detaily implementace a práci s konfiguratorem.

4.1.1 Nastavení konfiguratoru

Konfigurator poskytuje uživateli pokročilé možnosti přizpůsobení procesu vyhledávání souborů a odhalování duplicit. Umožňuje nastavit jak hloubku prohledávání adresářové struktury, tak i parametry pro detekci shodných souborů, čímž optimalizuje celkový výkon a přesnost vyhledávání. Tato flexibilita je klíčová pro efektivní správu datových úložišť větší velikosti.

Uživatel může definovat, do jaké hloubky adresářové struktury se má prohledávání provádět. Prohledávání lze omezit pouze na aktuální složku, kdy se prohledávají jen soubory přímo v ní. Dále je možné prohledávat pouze složky odpovídající předem definovanému filtru, například složky

obsahující určité klíčové slovo v názvu. Poslední možností je rekurzivní prohledávání, které systematicky prochází všechny podsložky a jejich obsah, čímž zaručuje kompletní pokrytí celé adresářové struktury od zvoleného kořenového adresáře směrem dolů.

Konfigurátor umožňuje aktivovat nebo deaktivovat kontrolu duplicitních souborů. Při zapnuté funkci jsou duplicitní soubory detekovány a zpracovány podle uživatelského nastavení. Program vrací seznam jedinečných souborů pro další zpracování a samostatný seznam identifikovaných duplicit s podrobnými informacemi. Detekce duplicit může probíhat ve dvou režimech, mezi kterými si uživatel může vybrat. Prvním režimem je porovnání na základě metadat, kdy se soubory porovnávají podle atributů, jako je velikost souboru a čas poslední modifikace. Tento přístup je velmi rychlý, ale nemusí vždy přesně detekovat všechny duplicity, například pokud jsou soubory stejné velikosti, ale mají odlišný obsah. Druhým režimem je hloubkové porovnání obsahu souborů, které porovnává skutečný obsah, například pomocí kontrolního součtu nebo binárního porovnání. Tento přístup je přesný, ale může být časově náročný, zejména při práci s velkým množstvím souborů nebo soubory o velké velikosti. [14]

4.1.2 Funkce programu

Tato sekce popisuje jednotlivé navržené funkce programu. Program nabízí možnosti vyhledávání, správu výsledků, podporu paralelního a asynchronního zpracování, generování výstupních zpráv a pokročilé nástroje pro automatizaci a analýzu dat. Díky těmto funkcím je možné pracovat i s velkým množstvím souborů a složitými scénáři zpracování dat.

4.1.2.1 Vyhledávací funkce

Vyhledávací funkce zahrnují několik možností. Program umožňuje filtrované vyhledávání, které poskytuje uživateli možnost přesně definovat, jaké soubory nebo složky mají být nalezeny na základě zadaných kritérií. Součástí je také rekurzivní procházení adresářové struktury. Uživatel může nastavit, zda se mají prohledávat i podadresáře, a případně omezit hloubku rekurze, což umožňuje lepší kontrolu nad rozsahem prohledávání. Program podporuje množinové operace s výsledky, jako je sjednocení, průnik a rozdíl. Díky tomu lze výsledky více různých dotazů snadno kombinovat nebo porovnávat.

4.1.2.2 Správa výsledků

Výsledky vyhledávání lze exportovat do různých formátů, konkrétně do formátu JSON nebo prostého textu. Program také umožňuje práci s proměnnými, kdy je možné ukládat výsledky do pojmenovaných množin pro pozdější využití.

Součástí správy výsledků je i historie operací. Program uchovává zásobník provedených akcí, umožňuje vracet jednotlivé operace zpět a eviduje časová razítka všech změn, což zajišťuje lepší přehled a kontrolu nad průběhem práce.

4.1.2.3 Paralelní zpracování

Program nabízí paralelní cyklus, který umožňuje automatické rozdělení souborů mezi dostupná jádra procesoru. Uživatel má také možnost ručně nastavit počet použitých jader. Pro paralelní zpracování lze využít jednoduchou syntaxi typu `pfor()` nebo `pfor_order()`, která zjednodušuje zápis paralelních úloh. Součástí je i možnost zpracovávat soubory po částech (tzv. chunkování¹), což je vhodné při práci s velkým množstvím dat.

4.1.2.4 Funkce pro výstupní zprávy

Mezi funkce pro výstupní zprávy patří generování reportů, které podporují použití šablon a vlastní formátování. Program dále nabízí e-mailové notifikace, které umožňují zasílání vlastních textových zpráv, přidání proměnných do těla e-mailu, automatické přiložení výsledků zpracování a možnost odeslat souhrnné hlášení po dokončení úlohy.

4.1.2.5 Pokročilé funkce

Pokročilé funkce zahrnují průběžné sledování stavu zpracování. Dále je zde analýza obsahu, která umožňuje fulltextové vyhledávání a podporu regulárních výrazů pro detailnější filtrování.

4.1.2.6 Technické detaily

Program podporuje dávkové zpracování (skriptování), což umožňuje automatizaci opakovaných úloh a snadné sestavení složitějších scénářů.

4.1.3 Architektura konfiguratoru

Architektura konfiguratoru je navržena modulárně, aby bylo možné jednotlivé části snadno udržovat, rozšiřovat a případně měnit podle potřeb uživatele. Každý modul se stará o specifickou oblast funkcionality a společně tvoří nástroj pro práci se soubory, správu historie, paralelní zpracování i čtení specializovaných datových formátů.

4.1.3.1 Modul pro zpracování příkazů

Slouží k interpretaci a validaci uživatelských příkazů, jejich směřování k příslušným modulům a také ke generování nápovědy a výstupních zpráv. Modul běží v samostatném vlákne, které je vytvořeno při spuštění nástroje. Díky tomu je možné zadávat nové příkazy během běhu operací a dotazovat se na aktuální stav bez blokování hlavního programu.

¹Chunk je menší, samostatně zpracovatelná část velkého datasetu, která umožňuje paralelní běh bez vzájemného ovlivňování jednotlivých bloků.

4.1.3.2 Modul pro vyhledávání souborů

Umožňuje rekurzivní procházení adresářové struktury a podporuje filtry podle jména (s využitím wildcard² a regulárních výrazů), velikosti (větší, menší, rovno, včetně podpory jednotek) a data modifikace či vytvoření. K optimalizovanému vyhledávání využívá knihovny `glob` a `pathlib`.

4.1.3.3 Modul pro historii

Zajišťuje uchovávání stavů programu, možnost navracení k předchozím stavům a evidenci časových razítek všech operací. Stavby představují například výběr souborů nebo výsledků, které je následně možné kombinovat.

4.1.3.4 Výstupní modul

Zodpovídá za formátování výsledků ve formě textových výstupů nebo JSON exportu, generování reportů a přípravu skriptů pro opakované použití. Součástí modulu je také funkce pro automatické odesílání výstupních zpráv e-mailem, včetně možnosti přiložení zpracovaných souborů nebo souhrnných logů. Modul rovněž umožňuje přidávání metadat (např. časová razítka, informace o použitých filtrech nebo konfiguraci běhu) do výsledného výstupu.

4.1.3.5 Modul pro paralelní zpracování

Implementuje funkce `pfor()` a `pfor_order()`, umožňuje distribuci výpočetní zátěže na více jader, podporuje sdílení globálního stavu mezi procesy a zajišťuje detekci a zpracování chyb v jednotlivých worker³ procesech.

4.1.3.6 Modul pro čtení Bionano souborů

Obsahuje specializované parsery pro formáty BNX, CMAP a XMAP. Součástí je validace vstupních souborů včetně kontroly hlavičky a ukládání molekul ve formě slovníků pro snadný přístup k jejich vlastnostem.

4.1.4 Pracovní postup

Použití konfigurátoru lze rozdělit do následujících kroků:

1. **Zadání kritérií:** Uživatel zadá kritéria vyhledávání, například přípony souborů (`.bnx`, `.cmap`), velikost (například nad 1 GB) nebo specifický adresář. K dispozici jsou také pokročilé filtry, například podle data modifikace nebo regulárních výrazů.

²Wildcard (zástupný znak) je speciální symbol (např. `*` nebo `_`) používaný při vyhledávání k reprezentaci libovolných znaků.

³Worker je samostatný vykonávací kontext (vlákno nebo proces), který zpracovává přiřazený chunk.

2. **Vyhledání souborů:** Konfigurátor vymezený adresář prohledá a zobrazí seznam souborů, které splňují zadané podmínky. Uživatel má možnost si seznam dále filtrovat nebo upravit.
3. **Výběr souborů:** Uživatel může vybrat konkrétní soubory nebo celé skupiny souborů, které chce zahrnout do dalšího zpracování. Tento výběr lze uložit jako stav pro pozdější použití nebo kombinovat s jinými výsledky.
4. **Příprava skriptů:** Vybrané soubory jsou uloženy do seznamu, který je předán k dalšímu zpracování. Následně jsou pro ně spuštěny připravené skripty s odpovídajícími parametry a příkazy pro výpočet.
5. **Zobrazení výsledků a generování reportu:** Po dokončení výpočtu je uživateli poskytnut výsledek ve formě textového výstupu nebo JSON souboru. Součástí je také možnost automatického vygenerování souhrnného reportu, který obsahuje přehled provedených operací, parametry a výsledky.

4.1.5 Výhody použití konfigurátoru

Hlavní přínosy konfigurátoru zahrnují zjednodušení celého procesu, protože automatizuje kroky, které by jinak vyžadovaly manuální zadávání. Dále minimalizuje chyby způsobené nesprávně zadanými cestami nebo parametry a zároveň poskytuje vysokou míru flexibility díky možnosti přizpůsobit kritéria vyhledávání a filtry podle konkrétních potřeb uživatele. Při správně definovaném příkazu je možné jej uložit a opakovaně používat pro podobné scénáře, což dále zvyšuje efektivitu.

4.1.6 Možné rozšíření konfigurátoru

Do budoucna lze uvažovat o využití databází pro ukládání metadat a rychlejší přístup k souborům. Další možností je vytvoření webového rozhraní s API, které by usnadnilo ovládání nástroje a zpřístupnilo jej širšímu okruhu uživatelů.

4.2 Implementace

Tato část popisuje konkrétní implementaci jednotlivých částí nástroje pro zpracování distribuovaných výpočtů pro soubory Bionano.

4.2.1 Použité technologie

Python

Python je interpretovaný, vysokoúrovňový programovací jazyk navržený s důrazem na jednoduchost a přehlednost zápisu. Cílem jeho návrhu bylo umožnit srozumitelný a čitelný kód i při řešení složitějších problémů. Jazyk je dynamicky typovaný a podporuje objektově orientovaný přístup.

V současnosti patří Python mezi nejpoužívanější jazyky napříč různými oblastmi informatiky od vědeckých výpočtů, datové analýzy a strojového učení až po vývoj webových aplikací, automatizaci nebo skripty. Díky rozsáhlé nabídce dostupných knihoven a modulů je možné jej efektivně využít také při práci s datovými formáty, pro správu systémových prostředků nebo při paralelním a distribuovaném zpracování úloh. [13]

Nástroj kromě standardních knihoven využívá také několik běžně dostupných externích modulů, které je možné snadno doinstalovat pomocí nástroje `pip`. Díky tomu zůstává implementace přehledná a snadno přenositelná na jiné systémy:

- `os` – pro práci se souborovým systémem
- `re` – podpora regulárních výrazů
- `glob` – vyhledávání souborů pomocí zástupných znaků
- `threading`, `multiprocessing` – paralelizace a zpracování úloh ve více vláknech nebo procesech
- `json` – čtení a export výsledků do JSON formátů
- `smtplib` – odesílání e-mailových notifikací po dokončení úloh

JSON

JSON (JavaScript Object Notation) je textový formát určený pro přenos a ukládání strukturovaných dat. Ačkoli je syntakticky odvozen od jazyka JavaScript, jedná se o formát nezávislý na konkrétním programovacím jazyce. Pro svou jednoduchost a čitelnost se JSON stal běžným prostředkem pro výměnu dat mezi systémy, zejména v oblasti webových rozhraní a služeb. [6]

Struktura JSON dokumentu je založena na několika základních prvcích: objekt (object), což je kolekce dvojic klíč–hodnota, kde klíče jsou vždy řetězce, pole (array), které představuje uspořádaný seznam hodnot a jeho prvky mohou být různého typu, a primitivní typy, mezi které patří řetězce, číselné hodnoty, logické hodnoty a hodnota `null`.

V jazyce Python je s tímto formátem možné pracovat pomocí vestavěného modulu `json`, který zajišťuje převod mezi běžnými datovými strukturami (např. `dict`, `list`) a jejich reprezentací v podobě JSON. Základní funkce zahrnují:

- `json.dump(obj, file)` – uloží JSON reprezentaci objektu do souboru.
- `json.load(file)` – načte JSON data ze souboru a převede je na odpovídající Python objekt.

Díky své jednoduchosti a široké podpoře v moderních programovacích jazycích je JSON považován za standardní formát pro přenos a ukládání dat mezi různými systémy.

4.2.2 Hlavní implementační koncepty

V této sekci jsou popsány klíčové principy návrhu a implementace nástroje. Shrnuje hlavní funkční bloky systému a přibližuje způsob jejich implementace z pohledu algoritmů i technických detailů. Tato část poskytuje kontext a vysvětluje jednotlivé komponenty, aby jim bylo možné lépe porozumět.

4.2.2.1 Vyhledávání a filtrování souborů

Systém implementuje pokročilé vyhledávání souborů pomocí příkazu `filter()`, který podporuje tři typy filtrů. Název souboru umožňuje využít wildcard výrazy (například `*.bnx`) a tímto způsobem lze hledat i podle přípony. Velikost souboru lze filtrovat pomocí operátorů (`<`, `>`, `=`) s podporou jednotek jako K, KB, MB nebo GB. Datum umožňuje filtrovat podle data modifikace nebo vytvoření s možností porovnání. Pokud je zapnutá funkce procházení adresářů, nástroj rekurzivně sestupuje adresářovou strukturou a vyhledává soubory i v podadresářích.

Syntaxe:

```
filter [name <vzor>] [size <operátor> <velikost>] \
[<modified/created> <operátor> <datum>]
```

Příklad použití:

```
filter name *.bnx size < 5 GB modified > 2023-01-01
```

Algoritmus pracuje v následujících krocích:

1. **Analýza příkazu:** Rozpozná a extrahuje jednotlivé filtry (pro název, velikost a datum) a validuje syntaxi.
2. **Globální vyhledávání:** Systém najde všechny odpovídající soubory pomocí optimalizované verze funkce `glob.glob()` a vrátí je jako seznam.

Níže uvedený kód 4.1 představuje způsob globálního vyhledávání souborů v adresáři. Nejprve načteme všechny položky odpovídající zadanému vzoru a následně z nich vyfiltrujeme pouze adresáře.

```
files = list(path.glob(name))
only_directories = [d for d in map(Path, files) if d.is_dir()]
```

Kód 4.1: Inicializace vyhledávání: načtení všech souborů a filtrace pouze adresářů

3. **Rekurzivní průchod:** Seznam se rozdělí na soubory a adresáře. Pro každý nalezený adresář se proces opakuje, pokud je aktivována rekurze.

Tento fragment kódu 4.2 demonstruje, jak se rekurzivně procházejí složky, pokud je tato funkce zapnutá. Pro každý adresář se volá funkce, která pokračuje v prohledávání podadresářů.

```

if global_variables.search_folders == 1:
    for folder in only_directories:
        temp_files, num_of_folders = search_folder(folder, commands)
        files.extend(temp_files)

```

Kód 4.2: Rekurzivní prohledávání složek, pokud je aktivní

4. **Aplikace filtrů:** Každý soubor postupně prochází všemi zadanými filtry, například kontrolou velikosti pomocí funkce `os.path.getsize()` a ověřením data modifikace či vytvoření prostřednictvím funkcí `os.path.getmtime()` a `os.path.getctime()`.
5. **Detekce duplicit:** Pokud se soubory liší v metadatech, jsou považovány za odlišné. Pokud však mají stejnou velikost, provádí se dodatečné porovnání podle data poslední modifikace. Pokud se shodují i v tomto ohledu, jsou vyhodnoceny jako duplicity.

Funkce `resolve_duplicity()` v kódu 4.3 prochází zadaný seznam souborů, porovnává je mezi sebou pomocí knihovny `filecmp` a rozděljuje je na unikátní soubory a duplicity. Soubor je považován za duplicitu, pokud nebyl dosud zpracován a shoduje se s jiným souborem. Výsledkem je dvojice seznamů. Seznam unikátních souborů a seznam nalezených duplicit.

```

def resolve_duplicity(files: list) -> tuple[list, list]:
    seen_files = set()
    duplicates, unique_files = []

    for current_file in files:
        if current_file not in seen_files:
            for other_file in files:
                if other_file not in seen_files and current_file != other_file and filecmp.cmp(
                    current_file, other_file, shallow=True):
                    duplicates.append(other_file)
                    seen_files.add(other_file)
            unique_files.append(current_file)
            seen_files.add(current_file)

    return unique_files, duplicates

```

Kód 4.3: Detekce duplicit s porovnáním metadat

4.2.2.2 Načítání strukturovaných Bionano dat

Funkce `load()` implementuje parser pro Bionano data ve formátu BNX, XMAP a CMAP s několika hlavními vlastnostmi. Podporuje hierarchické zpracování, kdy odděleně parsuje hlavičky, datové typy a samotná data. Zajišťuje také automatické přetypování hodnot podle specifikace uvedené

v hlavičkách. Obsahuje také detekci chyb, která umožňuje odhalit a nahlásit případné problémy ve vstupních souborech.

Syntaxe:

```
load(<cesta_k_souboru>)
```

Příklad použití:

```
data = load("/data/bionano/soubor.bnx")
```

Algoritmus pracuje v těchto fázích:

1. Validace souboru a zpracování hlaviček:

Algoritmus ověřuje, zda soubor existuje a jestli jsou k němu dostatečná přístupová práva. Kód 4.4 identifikuje různé typy hlaviček v souboru podle specifických prefixů. Řádky začínající `#h` jsou považovány za hlavní hlavičku, řádky `#f` definují datové typy a řádky s `#` následovaným číslicí představují vedlejší hlavičky. Hodnoty z těchto řádků jsou následně zpracovány a uloženy do slovníků.

```
if line.startswith('#h'): # Hlavni hlavicka
    headers["h"] = [column.strip() for column in line[2:].split("\t")]
elif line.startswith('#f'): # Typy dat
    data_types["f"] = [dtype.strip() for column in line[2:].split("\t")]
elif line[0] == "#" and line[1].isdigit(): # Vedlejsi hlavicky
    headers[line[1] + "h"] = [column.strip() for column in line[3:].split("\t")]
```

Kód 4.4: Identifikace typů hlaviček

2. Typová konverze dat:

Typová konverze zaručuje, že data načtená ze souboru odpovídají správným datovým typům definovaným v hlavičce. Hodnoty jsou dynamicky přetypovány na `float`, `int` nebo `bool`, případně zpracovány jako seznamy při použití zápisu `[N]`. Implementace tohoto mechanismu je uvedena v kódu 4.5.

```
if fields_data_types[i] == "float":
    value = float(value)
elif fields_data_types[i] == "int":
    value = int(value)
elif fields_data_types[i] == "bool":
    value = bool(value)
# Speciální případ pro pole hodnot (float)
elif '[N]' in fields_data_types[i]:
    data_row[field_name] = [float(v) for v in values[i:]]
```

Kód 4.5: Konverze hodnot podle hlavičky

3. **Sestavení výstupní struktury:** V kódu 4.6 se tvoří výstupní struktura ve formě slovníku (hashovací tabulky), která zajišťuje rychlý přístup a zároveň přehledné, jednotné uložení všech informací o souboru včetně názvu, cesty, komentářů, hlaviček, typů dat a samotných dat.

```
output_file = {  
    "filename": os.path.basename(processing_file),  
    "path": path,  
    "comments": comments,  
    "headers": headers,  
    "data_types": data_types,  
    "data": data  
}
```

Kód 4.6: Transformace do slovníku

Zpracování speciálních případů zahrnuje přeskočení nekompletních záznamů s varováním v případě chybějících hodnot a zpracování polí označených suffixem [N], tedy správné vyřešení těchto polí. Doplnující funkce zahrnují průběžné zobrazování stavu při dlouho běžících operacích, měření času zpracování a logování problémů.

Díky předchozí struktuře je možné soubor snadno načíst a přímo pracovat se surovými daty. Ukázka použití výsledné datové struktury je uvedena v kódu 4.7.

```
data = load("experiment.bnx")  
molecule_lengths = [item["Length"] for item in data.data]  
high_avgIntensity = [m for m in data["data"] if "AvgIntensity" in m and m["AvgIntensity"] > 2500]
```

Kód 4.7: Příklad práce s načtenými daty

Dva následující odstavce popisují klíčové aspekty načítání a zpracování Bionano dat, přičemž zdůrazňují jak strukturovaný přístup k parseru, tak zvládání speciálních případů a doplňujících funkcí, které zajišťují robustnost a přehlednost celého procesu.

Obrázek 4.1: Ukázka části vstupního BNX souboru v textové podobě. V horní části jsou vidět hlavičky (označené jako #h, #f), které popisují datové sloupce a jejich typy. Pod hlavičkami následují samotná surová data jednotlivých molekul. Každý řádek obsahuje například délku molekuly, průměrnou intenzitu nebo počet labelů. Tyto hodnoty jsou v textovém souboru odděleny tabulátory.

Obrázek 4.2: Vizualizace načtené a transformované datové struktury ve formátu JSON. Struktura obsahuje informace o názvu a cestě k souboru, komentáře, hlavičky jednotlivých sekcí a především pole data, které obsahuje přehledně zapsané hodnoty každé molekuly. Díky této reprezentaci je možné snadno přistupovat k jednotlivým atributům a dále s nimi pracovat v kódu.

```
#0h LabelChannel      MoleculeID      Length AvgIntensity      SNR NumberofLabels      OriginalMoleculeId      ScanNumber      ScanDirection      ChipId Flowcell      RunId      Column      StartFOV      StartX
StartY EndFOV EndX EndY GlobalScanNumber
#0f int int float float float int int int int string int int int int int int int int int int
#1h LabelChannel      LabelPositions[N]
#1f int float
#0h QualityScoreID      QualityScores[N]
#0f string float[N]
# Quality Score QX11: Label SNR for channel 1
# Quality Score QX12: Label Intensity for channel 1
0 1 874875.00 2199.94 118.90 125 1 1 -1 chips,SN_AE06H70JP3ZZFNMJ,Run_220eac77-51a8-4066-ab25-fdcecd81eae0,539583347 1 1 6 1 931 220 2 931 720 1
1 2439.00 4600.00 14732.00 44366.00 70823.00 77508.00 81105.00 91214.00 104140.00 106047.00 117372.00 119604.00 129939.00 132927.00 142374.00
145078.00 161826.00 166250.00 170838.00 178494.00 180994.00 186386.00 192935.00 203615.00 212823.00 237606.00 244518.00 247558.00 253014.00 256017.00
258035.00 261734.00 268742.00 276586.00 287704.00 291407.00 293934.00 297937.00 305085.00 310517.00 323964.00 329176.00 336101.00 341255.00 345657.00
347408.00 351654.00 352608.00 354702.00 357061.00 362257.00 367733.00 371223.00 373311.00 376962.00 389876.00 393695.00 395983.00 417628.00 419890.00
426254.00 436171.00 453793.00 455995.00 459046.00 464341.00 475691.00 477506.00 480241.00 484896.00 496182.00 498519.00 502302.00 516816.00 522222.00
533860.00 542674.00 550538.00 553313.00 556029.00 557205.00 565630.00 566979.00 574897.00 576916.00 581313.00 585679.00 603390.00 607414.00 613256.00
620880.00 627147.00 637817.00 638701.00 648961.00 648979.00 650303.00 652595.00 672808.00 675028.00 688179.00 695715.00 705176.00 709774.00 715576.00
720361.00 727996.00 732637.00 734888.00 753414.00 757973.00 760036.00 761429.00 771190.00 776904.00 778216.00 790242.00 796909.00 810585.00 828551.00
833535.00 834592.00 856094.00 858332.00 866596.00 874875.00
QX11 9.43 10.89 13.71 5.92 14.39 8.02 13.02 15.01 6.19 11.08 6.72 11.80 28.35 16.19 6.69 7.98 7.97 24.71 12.41 31.38 12.02 11.1
6 6.27 14.57 24.17 24.99 14.85 24.75 21.71 7.91 20.76 19.48 8.79 18.25 25.44 32.10 13.35 26.52 16.60 35.68 7.01 7.80 62.56 19.93
51.33 13.64 6.60 8.08 23.96 6.11 52.86 6.40 36.08 42.55 42.17 7.44 10.64 26.23 29.93 27.29 11.83 14.57 17.42 41.17 36.81 18.15 6
80 7.54 48.36 14.09 9.12 15.97 15.28 12.17 21.64 18.30 21.56 22.63 20.64 6.29 6.19 12.90 8.94 10.89 15.93 37.16 11.45 12.22 6.61
15.42 18.08 27.15 12.72 14.14 9.39 16.88 7.47 9.37 9.77 16.84 15.80 15.55 17.54 5.91 33.50 9.68 13.79 18.37 22.97 19.95 7.17 7
44 11.22 53.56 25.57 20.97 13.00 7.67 49.51 9.45 10.17 12.99 18.99 14.61 23.12
QX12 166.95 192.78 242.62 104.80 254.62 141.98 230.42 265.72 109.57 196.13 118.94 208.86 501.64 286.52 118.47 141.30 141.13 437.24 219.64 555.39 212.72 199
27 111.04 257.92 427.81 442.19 262.75 437.93 384.22 139.97 367.39 344.69 155.56 322.95 450.29 568.11 236.31 469.34 293.87 631.50 124.04 138.00 1107.23 352.64
908.34 241.37 116.74 142.94 424.02 108.08 935.53 113.22 638.46 752.99 746.28 126.39 188.27 464.12 529.67 483.03 209.31 257.82 308.36 728.59 651.45 321.22 120
29 133.52 855.76 249.30 161.39 282.67 270.40 215.35 383.05 323.81 381.49 400.52 365.28 111.35 109.52 228.22 158.13 192.66 281.94 657.69 202.56 216.23 116.93
272.94 319.99 480.47 225.08 250.24 166.22 298.69 132.22 165.76 172.92 298.04 279.57 275.16 310.40 104.58 592.90 171.33 244.10 183.49 406.58 353.05 126.83 131
62 198.61 947.95 452.45 371.18 230.06 135.73 876.24 167.21 179.99 229.93 336.11 258.65 409.16
0 2 691125.00 2281.71 156.23 123 2 1 -1 chips,SN_AE06H70JP3ZZFNMJ,Run_220eac77-51a8-4066-ab25-fdcecd81eae0,539583347 1 1 20 1 984 939 2 984 948 1
1 9466.00 12174.00 14650.00 17965.00 31561.00 49118.00 61510.00 66462.00 72642.00 77751.00 85596.00 99821.00 102772.00 107337.00 110582.00
115818.00 118528.00 124435.00 131275.00 140178.00 148157.00 149561.00 154066.00 156200.00 167334.00 176160.00 180076.00 191686.00 211175.00 214764.00
221019.00 224522.00 234405.00 238850.00 240688.00 244542.00 254115.00 256910.00 262124.00 275633.00 278726.00 282116.00 288085.00 290531.00 304416.00
310743.00 336756.00 346531.00 359056.00 360655.00 364510.00 365368.00 368196.00 374086.00 379020.00 384071.00 389259.00 391792.00 395828.00 402170.00
408904.00 410872.00 413349.00 416716.00 418572.00 427208.00 430406.00 434585.00 436421.00 447565.00 451335.00 453645.00 456834.00 463002.00 465707.00
473246.00 478963.00 483552.00 486262.00 491031.00 504807.00 513034.00 516266.00 519792.00 522078.00 530256.00 535797.00 543661.00 543179.00 548781.00
550663.00 553705.00 555935.00 557076.00 559111.00 562337.00 566707.00 570902.00 582834.00 592039.00 597519.00 599099.00 601767.00 605032.00 609306.00
610864.00 612698.00 614588.00 616739.00 618144.00 619981.00 621433.00 630681.00 641500.00 645905.00 649614.00 652330.00 656837.00 658941.00 662489.00
666218.00 668170.00 684169.00 691125.00
```

Obrázek 4.1: Původní struktura v textovém souboru

```
1 {
2   "filename": "/data/bionano",
3   "path": "2019-08-29_TBNNDDLPZXVNMJ_F2P1_8_30_2019_6_16_55_AM_Raw%Molecules.bmx",
4   "comments": [""],
22  },
23  "headers": {
24    "qh": [
25      "LabelChannel",
26      "MoleculeID",
27      "Length",
28      "AvgIntensity",
29      "SNR",
30      "NumberofLabels",
31      "OriginalMoleculeId",
32      "ScanNumber",
33      "ScanDirection",
34      "ChipId",
35      "Flowcell",
36      "RunId",
37      "Column",
38      "StartFOV",
39      "StartX",
40      "StartY",
41      "EndFOV",
42      "EndX",
43      "EndY",
44      "GlobalScanNumber"
45    ],
46    "1h": [
47      "LabelChannel",
48      "LabelPositions[N]"
49    ],
50    "qh": [
51      "QualityScoreID",
52      "QualityScores[N]"
53    ]
54  },
55  "data_types": {
56    "or": [
57      "int",
58      "int",
59      "float",
60      "float",
61      "float",
62      "int",
63      "int",
64      "int",
65      "int",
66      "string",
67      "int",
68      "int",
69      "int",
70      "int",
71      "int",
72      "int",
73      "int",
74      "int",
75      "int",
76      "int",
77      "int",
78      "int",
79      "int",
80      "int",
81      "int",
82      "int",
83      "int",
84      "int",
85      "int",
86      "int",
87      "int",
88      "int",
89      "int",
90      "int",
91      "int",
92      "int",
93      "int",
94      "int",
95      "int",
96      "int",
97      "int",
98      "int",
99      "int",
100     "int",
101     "int",
102     "int",
103     "int",
104     "int",
105     "int",
106     "int",
107     "int",
108     "int",
109     "int",
110     "int",
111     "int",
112     "int",
113     "int",
114     "int",
115     "int",
116     "int",
117     "int",
118     "int",
119     "int",
120     "int",
121     "int",
122     "int",
123     "int",
124     "int",
125     "int",
126     "int",
127     "int",
128     "int",
129     "int",
130     "int",
131     "int",
132     "int",
133     "int",
134     "int",
135     "int",
136     "int",
137     "int",
138     "int",
139     "int",
140     "int",
141     "int",
142     "int",
143     "int",
144     "int",
145     "int",
146     "int",
147     "int",
148     "int",
149     "int",
150     "int",
151     "int",
152     "int",
153     "int",
154     "int",
155     "int",
156     "int",
157     "int",
158     "int",
159     "int",
160     "int",
161     "int",
162     "int",
163     "int",
164     "int",
165     "int",
166     "int",
167     "int",
168     "int",
169     "int",
170     "int",
171     "int",
172     "int",
173     "int",
174     "int",
175     "int",
176     "int",
177     "int",
178     "int",
179     "int",
180     "int",
181     "int",
182     "int",
183     "int",
184     "int",
185     "int",
186     "int",
187     "int",
188     "int",
189     "int",
190     "int",
191     "int",
192     "int",
193     "int",
194     "int",
195     "int",
196     "int",
197     "int",
198     "int",
199     "int",
200     "int",
201     "int",
202     "int",
203     "int",
204     "int",
205     "int",
206     "int",
207     "int",
208     "int",
209     "int",
210     "int",
211     "int",
212     "int",
213     "int",
214     "int",
215     "int",
216     "int",
217     "int",
218     "int",
219     "int",
220     "int",
221     "int",
222     "int",
223     "int",
224     "int",
225     "int",
226     "int",
227     "int",
228     "int",
229     "int",
230     "int",
231     "int",
232     "int",
233     "int",
234     "int",
235     "int",
236     "int",
237     "int",
238     "int",
239     "int",
240     "int",
241     "int",
242     "int",
243     "int",
244     "int",
245     "int",
246     "int",
247     "int",
248     "int",
249     "int",
250     "int",
251     "int",
252     "int",
253     "int",
254     "int",
255     "int",
256     "int",
257     "int",
258     "int",
259     "int",
260     "int",
261     "int",
262     "int",
263     "int",
264     "int",
265     "int",
266     "int",
267     "int",
268     "int",
269     "int",
270     "int",
271     "int",
272     "int",
273     "int",
274     "int",
275     "int",
276     "int",
277     "int",
278     "int",
279     "int",
280     "int",
281     "int",
282     "int",
283     "int",
284     "int",
285     "int",
286     "int",
287     "int",
288     "int",
289     "int",
290     "int",
291     "int",
292     "int",
293     "int",
294     "int",
295     "int",
296     "int",
297     "int",
298     "int",
299     "int",
300     "int",
301     "int",
302     "int",
303     "int",
304     "int",
305     "int",
306     "int",
307     "int",
308     "int",
309     "int",
310     "int",
311     "int",
312     "int",
313     "int",
314     "int",
315     "int",
316     "int",
317     "int",
318     "int",
319     "int",
320     "int",
321     "int",
322     "int",
323     "int",
324     "int",
325     "int",
326     "int",
327     "int",
328     "int",
329     "int",
330     "int",
331     "int",
332     "int",
333     "int",
334     "int",
335     "int",
336     "int",
337     "int",
338     "int",
339     "int",
340     "int",
341     "int",
342     "int",
343     "int",
344     "int",
345     "int",
346     "int",
347     "int",
348     "int",
349     "int",
350     "int",
351     "int",
352     "int",
353     "int",
354     "int",
355     "int",
356     "int",
357     "int",
358     "int",
359     "int",
360     "int",
361     "int",
362     "int",
363     "int",
364     "int",
365     "int",
366     "int",
367     "int",
368     "int",
369     "int",
370     "int",
371     "int",
372     "int",
373     "int",
374     "int",
375     "int",
376     "int",
377     "int",
378     "int",
379     "int",
380     "int",
381     "int",
382     "int",
383     "int",
384     "int",
385     "int",
386     "int",
387     "int",
388     "int",
389     "int",
390     "int",
391     "int",
392     "int",
393     "int",
394     "int",
395     "int",
396     "int",
397     "int",
398     "int",
399     "int",
400     "int",
401     "int",
402     "int",
403     "int",
404     "int",
405     "int",
406     "int",
407     "int",
408     "int",
409     "int",
410     "int",
411     "int",
412     "int",
413     "int",
414     "int",
415     "int",
416     "int",
417     "int",
418     "int",
419     "int",
420     "int",
421     "int",
422     "int",
423     "int",
424     "int",
425     "int",
426     "int",
427     "int",
428     "int",
429     "int",
430     "int",
431     "int",
432     "int",
433     "int",
434     "int",
435     "int",
436     "int",
437     "int",
438     "int",
439     "int",
440     "int",
441     "int",
442     "int",
443     "int",
444     "int",
445     "int",
446     "int",
447     "int",
448     "int",
449     "int",
450     "int",
451     "int",
452     "int",
453     "int",
454     "int",
455     "int",
456     "int",
457     "int",
458     "int",
459     "int",
460     "int",
461     "int",
462     "int",
463     "int",
464     "int",
465     "int",
466     "int",
467     "int",
468     "int",
469     "int",
470     "int",
471     "int",
472     "int",
473     "int",
474     "int",
475     "int",
476     "int",
477     "int",
478     "int",
479     "int",
480     "int",
481     "int",
482     "int",
483     "int",
484     "int",
485     "int",
486     "int",
487     "int",
488     "int",
489     "int",
490     "int",
491     "int",
492     "int",
493     "int",
494     "int",
495     "int",
496     "int",
497     "int",
498     "int",
499     "int",
500     "int",
501     "int",
502     "int",
503     "int",
504     "int",
505     "int",
506     "int",
507     "int",
508     "int",
509     "int",
510     "int",
511     "int",
512     "int",
513     "int",
514     "int",
515     "int",
516     "int",
517     "int",
518     "int",
519     "int",
520     "int",
521     "int",
522     "int",
523     "int",
524     "int",
525     "int",
526     "int",
527     "int",
528     "int",
529     "int",
530     "int",
531     "int",
532     "int",
533     "int",
534     "int",
535     "int",
536     "int",
537     "int",
538     "int",
539     "int",
540     "int",
541     "int",
542     "int",
543     "int",
544     "int",
545     "int",
546     "int",
547     "int",
548     "int",
549     "int",
550     "int",
551     "int",
552     "int",
553     "int",
554     "int",
555     "int",
556     "int",
557     "int",
558     "int",
559     "int",
560     "int",
561     "int",
562     "int",
563     "int",
564     "int",
565     "int",
566     "int",
567     "int",
568     "int",
569     "int",
570     "int",
571     "int",
572     "int",
573     "int",
574     "int",
575     "int",
576     "int",
577     "int",
578     "int",
579     "int",
580     "int",
581     "int",
582     "int",
583     "int",
584     "int",
585     "int",
586     "int",
587     "int",
588     "int",
589     "int",
590     "int",
591     "int",
592     "int",
593     "int",
594     "int",
595     "int",
596     "int",
597     "int",
598     "int",
599     "int",
600     "int",
601     "int",
602     "int",
603     "int",
604     "int",
605     "int",
606     "int",
607     "int",
608     "int",
609     "int",
610     "int",
611     "int",
612     "int",
613     "int",
614     "int",
615     "int",
616     "int",
617     "int",
618     "int",
619     "int",
620     "int",
621     "int",
622     "int",
623     "int",
624     "int",
625     "int",
626     "int",
627     "int",
628     "int",
629     "int",
630     "int",
631     "int",
632     "int",
633     "int",
634     "int",
635     "int",
636     "int",
637     "int",
638     "int",
639     "int",
640     "int",
641     "int",
642     "int",
643     "int",
644     "int",
645     "int",
646     "int",
647     "int",
648     "int",
649     "int",
650     "int",
651     "int",
652     "int",
653     "int",
654     "int",
655     "int",
656     "int",
657     "int",
658     "int",
659     "int",
660     "int",
661     "int",
662     "int",
663     "int",
664     "int",
665     "int",
666     "int",
667     "int",
668     "int",
669     "int",
670     "int",
671     "int",
672     "int",
673     "int",
674     "int",
675     "int",
676     "int",
677     "int",
678     "int",
679     "int",
680     "int",
681     "int",
682     "int",
683     "int",
684     "int",
685     "int",
686     "int",
687     "int",
688     "int",
689     "int",
690     "int",
691     "int",
692     "int",
693     "int",
694     "int",
695     "int",
696     "int",
697     "int",
698     "int",
699     "int",
700     "int",
701     "int",
702     "int",
703     "int",
704     "int",
705     "int",
706     "int",
707     "int",
708     "int",
709     "int",
710     "int",
711     "int",
712     "int",
713     "int",
714     "int",
715     "int",
716     "int",
717     "int",
718     "int",
719     "int",
720     "int",
721     "int",
722     "int",
723     "int",
724     "int",
725     "int",
726     "int",
727     "int",
728     "int",
729     "int",
730     "int",
731     "int",
732     "int",
733     "int",
734     "int",
735     "int",
736     "int",
737     "int",
738     "int",
739     "int",
740     "int",
741     "int",
742     "int",
743     "int",
744     "int",
745     "int",
746     "int",
747     "int",
748     "int",
749     "int",
750     "int",
751     "int",
752     "int",
753     "int",
754     "int",
755     "int",
756     "int",
757     "int",
758     "int",
759     "int",
760     "int",
761     "int",
762     "int",
763     "int",
764     "int",
765     "int",
766     "int",
767     "int",
768     "int",
769     "int",
770     "int",
771     "int",
772     "int",
773     "int",
774     "int",
775     "int",
776     "int",
777     "int",
778     "int",
779     "int",
780     "int",
781     "int",
782     "int",
783     "int",
784     "int",
785     "int",
786     "int",
787     "int",
788     "int",
789     "int",
790     "int",
791     "int",
792     "int",
793     "int",
794     "int",
795     "int",
796     "int",
797     "int",
798     "int",
799     "int",
800     "int",
801     "int",
802     "int",
803     "int",
804     "int",
805     "int",
806     "int",
807     "int",
808     "int",
809     "int",
810     "int",
811     "int",
812     "int",
813     "int",
814     "int",
815     "int",
816     "int",
817     "int",
818     "int",
819     "int",
820     "int",
821     "int",
822     "int",
823     "int",
824     "int",
825     "int",
826     "int",
827     "int",
828     "int",
829     "int",
830     "int",
831     "int",
832     "int",
833     "int",
834     "int",
835     "int",
836     "int",
837     "int",
838     "int",
839     "int",
840     "int",
841     "int",
842     "int",
843     "int",
844     "int",
845     "int",
846     "int",
847     "int",
848     "int",
849     "int",
850     "int",
851     "int",
852     "int",
853     "int",
854     "int",
855     "int",
856     "int",
857     "int",
858     "int",
859     "int",
860     "int",
861     "int",
862     "int",
863     "int",
864     "int",
865     "int",
866     "int",
867     "int",
868     "int",
869     "int",
870     "int",
871     "int",
872     "int",
873     "int",
874     "int",
875     "int",
876     "int",
877     "int",
878     "int",
879     "int",
880     "int",
881     "int",
882     "int",
883     "int",
884     "int",
885     "int",
886     "int",
887     "int",
888     "int",
889     "int",
890     "int",
891     "int",
892     "int",
893     "int",
894     "int",
895     "int",
896     "int",
897     "int",
898     "int",
899     "int",
900     "int",
901     "int",
902     "int",
903     "int",
904     "int",
905     "int",
906     "int",
907     "int",
908     "int",
909     "int",
910     "int",
911     "int",
912     "int",
913     "int",
914     "int",
915     "int",
916     "int",
917     "int",
918     "int",
919     "int",
920     "int",
921     "int",
922     "int",
923     "int",
924     "int",
925     "int",
926     "int",
927     "int",
928     "int",
929     "int",
930     "int",
931     "int",
932     "int",
933     "int",
934     "int",
935     "int",
936     "int",
937     "int",
938     "int",
939     "int",
940     "int",
941     "int",
942     "int",
943     "int",
944     "int",
945     "int",
946     "int",
947     "int",
948     "int",
949     "int",
950     "int",
951     "int",
952     "int",
953     "int",
954     "int",
955     "int",
956     "int",
957     "int",
958     "int",
959     "int",
960     "int",
961     "int",
962     "int",
963     "int",
964     "int",
965     "int",
966     "int",
967     "int",
968     "int",
969     "int",
970     "int",
971     "int",
972     "int",
973     "int",
974     "int",
975     "int",
976     "int",
977     "int",
978     "int",
979     "int",
980     "int",
981     "int",
982     "int",
983     "int",
984     "int",
985     "int",
986     "int",
987     "int",
988     "int",
989     "int",
```

4.2.2.3 Paralelní zpracování příkazů

Pro urychlení výpočetně náročných operací je zvolen proces paralelního spuštění (paralelní cyklus) pomocí příkazů `pfor()` a `pfor_order()`. Oba příkazy umožňují využití více procesorových jader pro efektivní zpracování většího množství položek, jako jsou soubory nebo úlohy.

- `pfor()` — paralelizace bez garantovaného pořadí výsledků (rychlejší - menší režie)
- `pfor_order()` — garantuje pořadí výstupu podle vstupního seznamu

Syntaxe:

```
pfor(<funkce>, <list_souborů>, [<argumenty>], [num_cores=<počet_jader>])
```

Příklad použití:

```
results = pfor(load, list_of_files, arg1, arg2)
results = pfor_order(save, list_of_files, num_cores=64)
```

Algoritmus funguje následovně:

1. **Rozdělení vstupu:** Vstupní seznam je rozdělen do několika menších částí (chunků) podle počtu dostupných procesorových jader. Počet jader je automaticky detekován nebo zadán uživatelem. Rozdělení seznamu do chunků provádí kód 4.8.

```
num_cores = num_cores or multiprocessing.cpu_count()
chunks = np.array_split(items, num_cores)
```

Kód 4.8: Rozdělení vstupu do chunků podle počtu jader

2. **Inicializace sdílených dat:** Funkce `init_worker()` slouží k předání sdílených informací do každého procesu, jako je například aktuální cesta či globální proměnné potřebné pro výpočty. Tím zajistí, že všechny procesy mají přístup ke stejným datům bez nutnosti jejich opětovného načítání. (viz kód 4.9)

```
def init_worker(shared_data):
    global _shared_globals
    _shared_globals = shared_data
```

Kód 4.9: Inicializace sdílených dat ve workerech

3. **Vytvoření procesu:** Pomocí objektu `multiprocessing.Pool` se vytvoří skupina pracovních procesů odpovídající počtu dostupných jader (nebo zadanému počtu). Každý proces obdrží svůj chunk dat a zavolá zadanou funkci (`func`) pro každou položku v chunku. Výsledky jsou

asynchronně získávány metodou `imap_unordered()`, která vrací výsledky bez zachování původního pořadí. Funkce může kromě dat přijímat i další argumenty, sdílená data a identifikátor procesu (viz kód 4.10).

```
with multiprocessing.Pool(num_cores) as pool:
    results = []
    for result in pool.imap_unordered(worker_partial, chunks_with_ids):
        if result:
            results.extend(result if isinstance(result, list) else [result])
```

Kód 4.10: Základ paralelního zpracování pomocí funkce `pfor()`

4. **Stabilita:** Každý worker je chráněn vůči výjimkám. V případě chyby v konkrétním workeru je uložena jeho chyba a výpočet pokračuje bez přerušení.

Paralelní zpracování umožňuje výrazné zrychlení operací. Skvěle se hodí například při použití funkce `load()`, protože umožňuje zpracovávat více souborů současně. Lze jej však využít i pro libovolnou jinou funkci. Praktické využití je demonstrováno v sekci testování, konkrétně v části věnované rychlosti paralelizace (viz sekce 4.4.2).

4.3 Scénáře

V této sekci jsou popsány hlavní scénáře použití systému, které představují klíčové postupy zpracování dat a souborů. Každý scénář detailně popisuje aktéry, předpoklady, hlavní a alternativní průběh činností a očekávané výsledky, čímž poskytuje pohled na fungování systému v různých situacích. Scénáře jsou doplněné o diagramy znázorňující průběh jednotlivých procesů.

4.3.1 Use Case: Zpracování souborů pro analýzu

Tento use case popisuje kompletní proces práce se soubory určenými k následné analýze. Hlavními aktéry jsou uživatel, který definuje vstupní parametry a dohlíží na průběh, a samotný software, jenž zajišťuje samotné vyhledávání souborů.

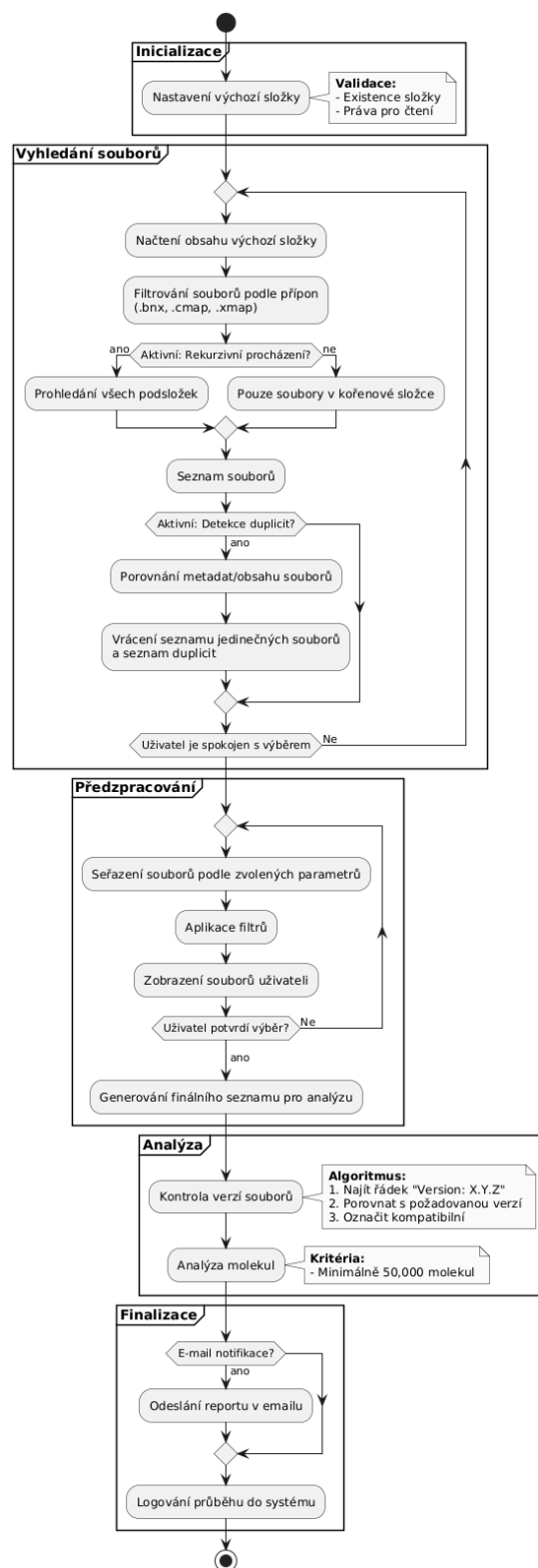
Proces začíná inicializací, kdy uživatel definuje výchozí složku obsahující vstupní soubory. Systém následně provede validaci této složky, konkrétně ověření její existence a kontrolu přístupových práv nezbytných pro čtení dat. Po úspěšném ověření přistupuje systém k vyhledání souborů, přičemž načte obsah určené složky a filtruje soubory na základě předem definovaných přípon. V případě aktivace rekurzivního prohledávání jsou jednotlivé podsložky procházeny stejným způsobem a jejich obsah je ukládán pro další zpracování. Pokud je povolena detekce duplicit, systém zaznamenává duplicitní soubory odděleně a odlišuje je od jedinečných souborů. Výsledkem této fáze je souhrnný seznam souborů, které splňují všechna zadaná kritéria, a seznam duplicit, na jejichž základě lze rozhodnout, co s nimi dále provést.

Následuje fáze předzpracování, ve které systém soubory seřadí dle zvolených parametrů, například podle velikosti nebo data poslední modifikace. Současně jsou na soubory aplikovány definované filtry a uživateli je zobrazena jejich náhledová podoba k finálnímu potvrzení. Po potvrzení výběru přechází systém k samotné analýze. V této části probíhá kontrola verzí jednotlivých souborů a analýza molekul na základě předem stanovených kritérií, například minimálního požadovaného počtu molekul nebo korektních vlastností. Soubory, které splní všechny podmínky, jsou předány dál k závěrečné fázi.

V závěrečné fázi je automaticky odeslána souhrnná zpráva o analýze prostřednictvím e-mailu. Součástí tohoto kroku je také kompletní záznam všech provedených operací, který je přiložen jak v souhrnné zprávě, tak uložen v logovacím souboru. To umožňuje dodatečnou revizi a kontrolu celého procesu.

Pokud uživatel není spokojen s výsledným výběrem souborů, systém umožňuje opakované vyhledání a úpravu filtrů až do dosažení požadovaného stavu. Výsledkem celého procesu je tedy nejen získání vyfiltrovaných a analyzovaných souborů, ale také kompletní zpráva doručená uživateli, doplněná uloženým záznamem průběhu analýzy.

Přehledný průběh celého procesu je znázorněn v activity diagramu na obrázku 4.3.



Obrázek 4.3: Activity diagram vyhledání souborů pro analýzu

4.3.2 Use Case: Vyhledání souborů se stejným názvem a úspěšným dokončením

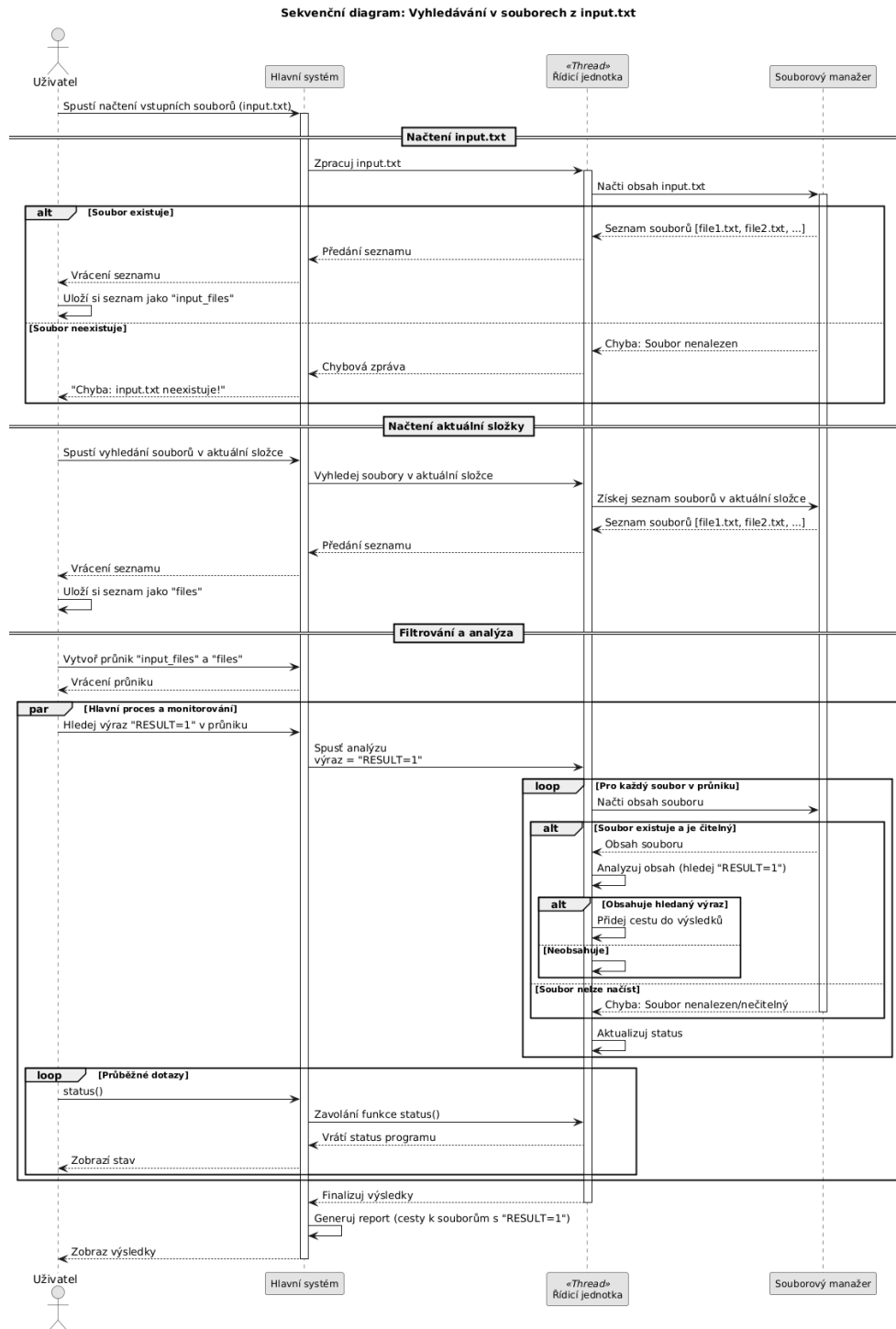
Další use case popisuje proces analýzy souborů, které jsou načteny ze vstupního souboru obsahujícího seznam již dříve vyhledaných souborů. Cílem je zjistit, zda jednotlivé soubory obsahují specifický řetězec indikující jejich úspěšné dokončení. Hlavními aktéry jsou uživatel, program, řídicí jednotka a souborový manažer.

Proces začíná načtením vstupního souboru, který obsahuje seznam souborů určených k dalšímu zpracování. Uživatel nejprve zadá cestu k tomuto vstupnímu souboru. Systém následně ověří jeho existenci a čitelnost a poté z něj načte seznam souborů. Pokud vstupní soubor neexistuje nebo není čitelný, systém zobrazí chybové hlášení a proces ukončí. V dalším kroku dochází k porovnání těchto souborů s aktuálně vyhledanými soubory v systému, přičemž cílem je identifikovat pouze ty soubory, které odpovídají zadaným názvům nebo cestám.

Po úspěšném načtení přechází proces k analýze obsahu jednotlivých souborů. Pro každý soubor v seznamu systém otevře soubor pro čtení a prohledává jeho obsah se zaměřením na výskyt řetězce "RESULT=1", který slouží jako indikátor úspěšného dokončení příslušné úlohy. Pokud je tento řetězec nalezen, systém jej zaznamená a uloží informaci o úspěšném dokončení daného souboru. Výsledkem této fáze je přehled všech nalezených souborů, které splňují kritéria.

V závěrečné části procesu systém vytvoří souhrnný report, ve kterém uvede seznam analyzovaných souborů, počet souborů obsahujících požadovaný řetězec a konkrétní umístění nalezených výsledků. Tento report je následně předán uživateli, který tak získá přehled o stavu všech analyzovaných souborů.

Výsledkem celého procesu je tedy přehledná zpráva o souborech, které obsahují indikaci úspěšného dokončení, a také podrobný log celého průběhu analýzy uložený v systému. Souhrnný průběh procesu je znázorněn v sekvenčním diagramu na obrázku 4.4.



Obrázek 4.4: Sekvenční diagram scénáře vyhledávání v souborech

4.3.3 Use Case: Paralelní zpracování seznamu položek připraveným skriptem

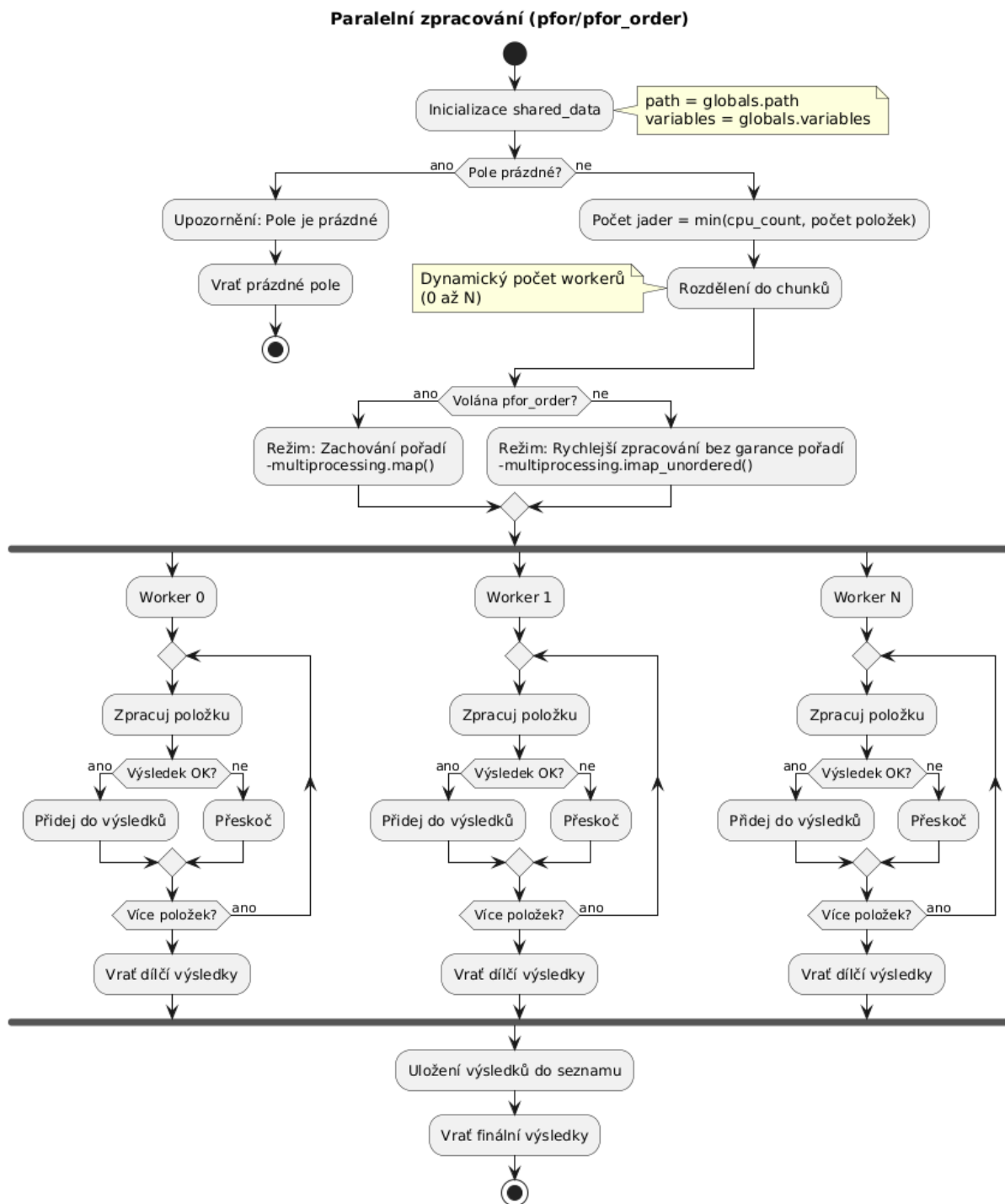
Poslední use case se zabývá procesem paralelního zpracování vstupního seznamu položek s využitím předem definované funkce nebo skriptu. Hlavními aktéry jsou uživatel, systém, řídicí jednotka a pracovní vlákna. Záměrem celého procesu je rozdělení seznamu na menší části, takzvané chunky, které jsou následně zpracovávány paralelně na vícejádrovém procesoru. Po dokončení všech dílčích úloh jsou výsledky sloučeny a předány uživateli ve formě finálního souhrnného výstupu.

Pro úspěšné provedení tohoto scénáře je nezbytné, aby existoval validní skript nebo funkce, která bude použita při zpracování jednotlivých položek. Dále je vyžadován vstupní seznam položek, například ve formě výběru Bionano souborů. Systém musí zároveň disponovat dostatečnými výpočetními prostředky, zejména pamětí a procesorovým výkonem, které umožní efektivní paralelní běh.

Celý proces začíná přípravou vstupů, kdy uživatel určí konkrétní skript nebo funkci a poskytne vstupní seznam položek. Systém provede kontrolu existence a čitelnosti zvoleného skriptu a ověří připravenost seznamu pro další zpracování. Po splnění všech podmínek následuje inicializace paralelního běhu. Uživatel spustí zpracování a systém předá požadavek řídicí jednotce. Ta rozdělí seznam na několik dílčích částí podle počtu dostupných jader, čímž umožní efektivní využití hardwarových prostředků.

Každý vytvořený chunk je následně zpracován samostatným pracovním vláknem. Každé vlákno načte přidělené položky a pro každou z nich spustí definovaný skript nebo funkci. Výsledky zpracování jednotlivých položek jsou ukládány jako mezivýsledky, které jsou po dokončení celého chunku předány zpět řídicí jednotce. Jakmile všechna pracovní vlákna dokončí svoji činnost, řídicí jednotka shromáždí mezivýsledky a zajistí jejich sloučení do finálního výstupu. Tento souhrnný výsledek je následně předán uživateli ve formě reportu obsahujícího výsledky zpracování jednotlivých položek. Součástí je rovněž log záznam celého paralelního běhu, zahrnující informace o časech spuštění a ukončení jednotlivých vláken a případně zaznamenané chyby.

V případě, že skript nebo funkce neprojde úvodní validací, systém okamžitě informuje uživatele prostřednictvím chybového hlášení a proces ukončí. Stejný postup nastává také v případě, že vstupní seznam položek neexistuje nebo není čitelný. Pokud dojde k chybě během samotného paralelního běhu, například selhání některého pracovního vlákna z důvodu výjimky nebo nečitelných dat, řídicí jednotka to zaznamená. Zpracování ostatních částí tím však není ovlivněno a pokračuje dále. Celý proces paralelního zpracování, včetně větvení a správy workerů, je přehledně zachycen v activity diagramu na obrázku 4.5.



Obrázek 4.5: Activity diagram paralelního zpracování

4.4 Testování

Testování odhalilo několik významných překážek, které komplikovaly používání nástroje. Jedním z hlavních problémů se ukázala být nutnost manuální instalace potřebných knihoven a závislostí na serveru (pravděpodobně kvůli zavřenému portu, který potřebuje manažer knihoven), což prodlužovalo proces přípravy testovacího prostředí a vyžadovalo další konfigurační kroky pro jednoduché spouštění na serveru.

Během testování se objevily významné rozdíly v chování aplikace mezi Windows a Linux prostředím. Hlavním problémem byla různá interpretace cest k souborům, která se povedla vyřešit díky normalizaci cest.

Následující sekce představují praktické testování výkonnosti systému při zpracování různých typů a velikostí souborů. Testování probíhalo jak na lokálním počítači, tak na serverovém prostředí, což umožnilo porovnání chování systému v různých podmínkách. V této části jsou prezentovány výsledky měření rychlosti výběru souborů, vlivu paralelizace a efektivity přístupu k načteným datům.

4.4.1 Rychlost výběru souborů

Funkce pro výběr a filtraci souborů funguje správně a efektivně. Systém spolehlivě filtruje různé typy souborů a umožňuje aplikovat i pokročilé filtrační podmínky bez výrazného dopadu na rychlost. Testování proběhlo na lokálním počítači i v serverovém prostředí.

Na základě naměřených hodnot nebylo potvrzeno, že použití SSD disku vždy vede k vyšší rychlosti výběru souborů. Výsledky ukázaly, že hlavním faktorem ovlivňujícím rychlost je především celkový počet souborů. V některých případech dosáhl HDD dokonce vyšší rychlosti než SSD, což lze považovat za odchylku.

Na obrázku 4.6 je znázorněno využití funkce `filter()` s nastavením pro vyhledávání všech souborů s příponou `bnx`, jejichž velikost přesahuje 1 GB a které byly modifikovány po datu 1. 1. 2020. Výsledkem bylo více než 200 souborů uložených v různých složkách. Tento výběr lze dále zúžit nebo použít pro spojení s jiným výběrem. Při pokračování jsou tyto soubory následně načteny a zpracovány.

```

/data1/bionano-old/compute/jobs >> filter name *.bnx size > 1 GB modified > 1/1/2020
Progress: [=====] 100%
Found 211 files and 0 folders

```

File	Size	Modified Time	Created Time
/data1/bionano-old/compute/jobs/264/RawMolecules.bnx	2.33 GB	14/01/2020	07/03/2023
/data1/bionano-old/compute/jobs/264/output/all.bnx	2.33 GB	14/01/2020	07/03/2023
/data1/bionano-old/compute/jobs/264/output/all_sorted.bnx	2.06 GB	14/01/2020	07/03/2023
/data1/bionano-old/compute/jobs/264/output/data/auto_noise/autoNoise1_rescaled.bnx	1.93 GB	14/01/2020	07/03/2023
/data1/bionano-old/compute/jobs/265/RawMolecules.bnx	2.33 GB	15/01/2020	07/03/2023
/data1/bionano-old/compute/jobs/265/output/all.bnx	2.33 GB	15/01/2020	07/03/2023
/data1/bionano-old/compute/jobs/265/output/all_sorted.bnx	2.06 GB	15/01/2020	07/03/2023
/data1/bionano-old/compute/jobs/265/output/data/auto_noise/autoNoise1_rescaled.bnx	1.93 GB	15/01/2020	07/03/2023
/data1/bionano-old/compute/jobs/266/RawMolecules.bnx	2.33 GB	16/01/2020	07/03/2023
/data1/bionano-old/compute/jobs/266/output/all.bnx	2.33 GB	16/01/2020	07/03/2023
/data1/bionano-old/compute/jobs/266/output/all_sorted.bnx	2.06 GB	16/01/2020	07/03/2023
/data1/bionano-old/compute/jobs/266/output/data/auto_noise/autoNoise1_rescaled.bnx	1.93 GB	16/01/2020	07/03/2023
/data1/bionano-old/compute/jobs/267/startNR.20011709592115200975/GM24149_DLS_hg19_-_molecules_20180413_GM24149.bnx	1.10 GB	17/01/2020	07/03/2023
/data1/bionano-old/compute/jobs/268/20180413_GM24149.bnx	1.10 GB	17/01/2020	07/03/2023
/data1/bionano-old/compute/jobs/268/output/all.bnx	1.10 GB	17/01/2020	07/03/2023
/data1/bionano-old/compute/jobs/268/output/all_sorted.bnx	1.05 GB	17/01/2020	07/03/2023
/data1/bionano-old/compute/jobs/268/output/contigs/auto_noise/autoNoise8_rescaled.bnx	1.01 GB	17/01/2020	07/03/2023
/data1/bionano-old/compute/jobs/268/output/contigs/auto_noise/autoNoise1_rescaled.bnx	1.00 GB	17/01/2020	07/03/2023
/data1/bionano-old/compute/jobs/269/GM24149_DLS_hg19_-_molecules_20180413_GM24149.bnx	1.10 GB	17/01/2020	07/03/2023
/data1/bionano-old/compute/jobs/269/output/all.bnx	1.10 GB	17/01/2020	07/03/2023
/data1/bionano-old/compute/jobs/269/output/contigs/auto_noise/autoNoise8_rescaled.bnx	1.01 GB	17/01/2020	07/03/2023
/data1/bionano-old/compute/jobs/270/GM24149_DLS_hg19_-_molecules_20180413_GM24149.bnx	1.10 GB	17/01/2020	07/03/2023
/data1/bionano-old/compute/jobs/270/output/all.bnx	1.10 GB	18/01/2020	07/03/2023
/data1/bionano-old/compute/jobs/270/output/all_sorted.bnx	1.05 GB	17/01/2020	07/03/2023
/data1/bionano-old/compute/jobs/270/output/contigs/auto_noise/autoNoise8_rescaled.bnx	1.01 GB	17/01/2020	07/03/2023
/data1/bionano-old/compute/jobs/270/output/contigs/auto_noise/autoNoise1_rescaled.bnx	1.00 GB	17/01/2020	07/03/2023
/data1/bionano-old/compute/jobs/271/startNR.200124114808151869595/S51_merge_RawMolecules.bnx	13.39 GB	24/01/2020	07/03/2023
/data1/bionano-old/compute/jobs/381/S51_merge_RawMolecules.bnx	13.39 GB	25/01/2020	07/03/2023
/data1/bionano-old/compute/jobs/381/output/all.bnx	13.39 GB	25/01/2020	07/03/2023
/data1/bionano-old/compute/jobs/381/output/all_sorted.bnx	6.77 GB	25/01/2020	07/03/2023
/data1/bionano-old/compute/jobs/381/output/contigs/auto_noise/autoNoise8_rescaled.bnx	4.71 GB	25/01/2020	07/03/2023
/data1/bionano-old/compute/jobs/381/output/contigs/auto_noise/autoNoise1_rescaled.bnx	3.50 GB	25/01/2020	07/03/2023
/data1/bionano-old/compute/jobs/311/S51_merge_RawMolecules.bnx	13.39 GB	27/01/2020	07/03/2023
/data1/bionano-old/compute/jobs/311/output/all.bnx	13.39 GB	27/01/2020	07/03/2023
/data1/bionano-old/compute/jobs/311/output/all_sorted.bnx	7.41 GB	27/01/2020	07/03/2023
/data1/bionano-old/compute/jobs/311/output/data/auto_noise/autoNoise1_rescaled.bnx	4.67 GB	27/01/2020	07/03/2023
/data1/bionano-old/compute/jobs/329/startNR.200220100511382142674/GM24385_DLS_hg19_-_molecules_20180413_GM24385.bnx	1.57 GB	20/02/2020	07/03/2023
/data1/bionano-old/compute/jobs/330/GM24385_DLS_hg19_-_molecules_20180413_GM24385.bnx	1.57 GB	20/02/2020	07/03/2023
/data1/bionano-old/compute/jobs/330/output/all.bnx	1.57 GB	20/02/2020	07/03/2023
/data1/bionano-old/compute/jobs/330/output/contigs/auto_noise/autoNoise8_rescaled.bnx	1.29 GB	20/02/2020	07/03/2023
/data1/bionano-old/compute/jobs/330/output/contigs/auto_noise/autoNoise1_rescaled.bnx	1.23 GB	20/02/2020	07/03/2023
/data1/bionano-old/compute/jobs/369/startNR.200702134712508032175/GM06226.bnx	2.81 GB	02/07/2020	07/03/2023
/data1/bionano-old/compute/jobs/370/startNR.200702135514751166366/GM08331.bnx	2.64 GB	02/07/2020	07/03/2023
/data1/bionano-old/compute/jobs/372/startNR.200702142006064580810/GM09888.bnx	2.28 GB	02/07/2020	07/03/2023
/data1/bionano-old/compute/jobs/375/startNR.200702144551995449434/GM11420.bnx	3.46 GB	02/07/2020	07/03/2023
/data1/bionano-old/compute/jobs/377/startNR.200702145156049631998/GM24143.bnx	1.38 GB	02/07/2020	07/03/2023
/data1/bionano-old/compute/jobs/378/startNR.200702145521130869104/GM09888.bnx	2.28 GB	07/07/2020	07/03/2023
/data1/bionano-old/compute/jobs/393/startNR.210315082631160205122/S33_-_FC1.bnx	2.39 GB	15/03/2021	07/03/2023
/data1/bionano-old/compute/jobs/394/startNR.210315083053760998764/S34_-_FC2.bnx	3.14 GB	15/03/2021	07/03/2023
/data1/bionano-old/compute/jobs/395/S33_-_FC1.bnx	2.39 GB	15/03/2021	07/03/2023
/data1/bionano-old/compute/jobs/395/output/all.bnx	2.39 GB	15/03/2021	07/03/2023
/data1/bionano-old/compute/jobs/395/output/contigs/auto_noise/autoNoise8_rescaled.bnx	1.36 GB	15/03/2021	07/03/2023
/data1/bionano-old/compute/jobs/395/output/contigs/auto_noise/autoNoise1_rescaled.bnx	1.27 GB	15/03/2021	07/03/2023
/data1/bionano-old/compute/jobs/396/S34_-_FC2.bnx	3.14 GB	15/03/2021	07/03/2023
/data1/bionano-old/compute/jobs/396/output/all.bnx	3.14 GB	15/03/2021	07/03/2023
/data1/bionano-old/compute/jobs/396/output/contigs/auto_noise/autoNoise8_rescaled.bnx	2.06 GB	15/03/2021	07/03/2023
/data1/bionano-old/compute/jobs/396/output/contigs/auto_noise/autoNoise1_rescaled.bnx	2.05 GB	15/03/2021	07/03/2023
/data1/bionano-old/compute/jobs/405/startNR.220319203833495557890/hg19.bnx	1.10 GB	19/03/2022	07/03/2023
/data1/bionano-old/compute/jobs/406/hg19.bnx	1.10 GB	19/03/2022	07/03/2023
/data1/bionano-old/compute/jobs/406/output/all.bnx	1.10 GB	19/03/2022	07/03/2023
/data1/bionano-old/compute/jobs/406/output/contigs/auto_noise/autoNoise8_rescaled.bnx	1.01 GB	19/03/2022	07/03/2023
/data1/bionano-old/compute/jobs/414/startNR.220405084237008271243/GM24385_DLS_hg38_GM24385_DLS_hg19_-_molecules_20180413_GM24385.bnx	1.57 GB	05/04/2022	07/03/2023
/data1/bionano-old/compute/jobs/415/GM24385_DLS_hg38_GM24385_DLS_hg19_-_molecules_20180413_GM24385.bnx	1.57 GB	05/04/2022	07/03/2023
/data1/bionano-old/compute/jobs/415/output/all.bnx	1.57 GB	05/04/2022	07/03/2023

Obrázek 4.6: Použití funkce filter()

Tabulka 4.1 ukazuje porovnání rychlosti výběru souborů v různých prostředích a typech disků. Sloupec „Počet souborů“ uvádí množství zpracovaných souborů, zatímco „Filtreační podmínky“ specifikují kritéria, podle kterých byly soubory vybírány. Čas výběru udává dobu potřebnou k aplikaci filtru na daný počet souborů, vyjádřenou v sekundách. Poslední sloupec prezentuje rychlost zpracování, tedy počet vybraných souborů za sekundu. Lze vidět, že rychlost výběru souborů není výrazně ovlivněna složitostí filtračních podmínek. I při přidání dalších kritérií, jako je velikost nebo datum vytvoření/modifikace, se čas výběru a rychlost zpracování výrazně nemění.

Tabulka 4.1: Porovnání rychlosti výběru souborů

Prostředí	Typ disku	Počet souborů	Filtrační podmínky	Čas výběru [s]	Rychlost [souborů/s]
Lokální PC	SSD	213 347	name *.bnx size < 5 GB modified > 10/01/2015	7,39	28 869
Lokální PC	SSD	452 785	name all* size > 100 MB created > 20/05/2018	16,35	27 693
Lokální PC	HDD	323 692	name *.bnx size > 100 MB created > 20/05/2018	9,40	34 435
Server	-	405 421	name *.bnx	9,65	42 012
Server	-	405 421	name *.bnx size > 5 GB	9,96	40 704
Server	-	405 421	name *.bnx size > 5 GB created > 05/10/2020	10,48	38 685

4.4.2 Rychlost paralelizace

Paralelní zpracování pomocí modulu `multiprocessing` přineslo výrazné zrychlení, efektivita však silně závisela na velikosti načítaných souborů a náročnosti spouštěného skriptu. U jednodušších operací s malými soubory vedla paralelizace k horším výsledkům než sekvenční zpracování kvůli režii spojené s vytvářením procesů. U velkých souborů se paralelizace projevila jako efektivní řešení.

Na obrázku 4.7 je pro příklad rozdělení načítání souborů mezi 20 procesorových jader, které přineslo výrazné zrychlení celkového procesu. Zatímco původní sekvenční implementace trvala 3 minuty a 42 sekund, paralelní verze tuto dobu snížila na 46 sekund.

```
/data1/bionano-old/compute/jobs/10/output/all_9_of_28.bnx 70.33 MB 10/01/2019 06/03/2023
/data1/bionano-old/compute/jobs/10/output/all_3_of_28.bnx 70.35 MB 10/01/2019 06/03/2023
/data1/bionano-old/compute/jobs/10/output/all_5_of_28.bnx 70.36 MB 10/01/2019 06/03/2023
/data1/bionano-old/compute/jobs/10/output/all_10_of_28.bnx 70.39 MB 10/01/2019 06/03/2023
/data1/bionano-old/compute/jobs/10/output/all_27_of_28.bnx 70.44 MB 10/01/2019 06/03/2023
/data1/bionano-old/compute/jobs/10/output/all_4_of_28.bnx 70.48 MB 10/01/2019 06/03/2023
/data1/bionano-old/compute/jobs/10/output/all_11_of_28.bnx 70.49 MB 10/01/2019 06/03/2023
/data1/bionano-old/compute/jobs/10/output/all_2_of_28.bnx 70.60 MB 10/01/2019 06/03/2023
/data1/bionano-old/compute/jobs/10/output/all_18_of_28.bnx 70.61 MB 10/01/2019 06/03/2023
/data1/bionano-old/compute/jobs/10/output/all_26_of_28.bnx 70.62 MB 10/01/2019 06/03/2023
/data1/bionano-old/compute/jobs/10/output/all_22_of_28.bnx 70.67 MB 10/01/2019 06/03/2023
/data1/bionano-old/compute/jobs/10/output/all_25_of_28.bnx 70.72 MB 10/01/2019 06/03/2023
/data1/bionano-old/compute/jobs/10/output/all_6_of_28.bnx 70.78 MB 10/01/2019 06/03/2023
/data1/bionano-old/compute/jobs/10/output/all_20_of_28.bnx 70.78 MB 10/01/2019 06/03/2023
/data1/bionano-old/compute/jobs/10/output/all_21_of_28.bnx 70.79 MB 10/01/2019 06/03/2023
/data1/bionano-old/compute/jobs/10/output/all_13_of_28.bnx 70.80 MB 10/01/2019 06/03/2023
/data1/bionano-old/compute/jobs/10/output/all_14_of_28.bnx 70.84 MB 10/01/2019 06/03/2023
/data1/bionano-old/compute/jobs/10/output/all_16_of_28.bnx 70.85 MB 10/01/2019 06/03/2023
-----
/data1/bionano-old/compute/jobs >> add *
Added 20 files
Added files (20):
/data1/bionano-old/compute/jobs/10/output/all_28_of_28.bnx
/data1/bionano-old/compute/jobs/10/output/all_7_of_28.bnx
/data1/bionano-old/compute/jobs/10/output/all_9_of_28.bnx
/data1/bionano-old/compute/jobs/10/output/all_3_of_28.bnx
/data1/bionano-old/compute/jobs/10/output/all_5_of_28.bnx
/data1/bionano-old/compute/jobs/10/output/all_10_of_28.bnx
/data1/bionano-old/compute/jobs/10/output/all_27_of_28.bnx
/data1/bionano-old/compute/jobs/10/output/all_4_of_28.bnx
/data1/bionano-old/compute/jobs/10/output/all_11_of_28.bnx
/data1/bionano-old/compute/jobs/10/output/all_2_of_28.bnx
/data1/bionano-old/compute/jobs/10/output/all_18_of_28.bnx
/data1/bionano-old/compute/jobs/10/output/all_26_of_28.bnx
/data1/bionano-old/compute/jobs/10/output/all_22_of_28.bnx
/data1/bionano-old/compute/jobs/10/output/all_25_of_28.bnx
/data1/bionano-old/compute/jobs/10/output/all_6_of_28.bnx
/data1/bionano-old/compute/jobs/10/output/all_20_of_28.bnx
/data1/bionano-old/compute/jobs/10/output/all_21_of_28.bnx
/data1/bionano-old/compute/jobs/10/output/all_13_of_28.bnx
/data1/bionano-old/compute/jobs/10/output/all_14_of_28.bnx
/data1/bionano-old/compute/jobs/10/output/all_16_of_28.bnx
/data1/bionano-old/compute/jobs >> loaded_files = pfor(load, added)
Starting 20 cores...
Variable "loaded_files" saved
/data1/bionano-old/compute/jobs >> avg_snrs = pfor(average_snrs, loaded_files)
Starting 20 cores...
Variable "avg_snrs" saved
/data1/bionano-old/compute/jobs >> print(avg_snrs)
[['/data1/bionano-old/compute/jobs/10/output/all_10_of_28.bnx', 80.81353562338298], ['/data1/bionano-old/compute/jobs/10/output/all_14_of_28.bnx', 80.09329771345664], ['/data1/bionano-old/compute/jobs/10/output/all_9_of_28.bnx', 80.9139961947061], ['/data1/bionano-old/compute/jobs/10/output/all_4_of_28.bnx', 81.14570310692228], ['/data1/bionano-old/compute/jobs/10/output/all_26_of_28.bnx', 75.2611636761396], ['/data1/bionano-old/compute/jobs/10/output/all_27_of_28.bnx', 74.31648390808086], ['/data1/bionano-old/compute/jobs/10/output/all_2_of_28.bnx', 81.58945581525553], ['/data1/bionano-old/compute/jobs/10/output/all_25_of_28.bnx', 75.6061423252498], ['/data1/bionano-old/compute/jobs/10/output/all_3_of_28.bnx', 82.11781737100134], ['/data1/bionano-old/compute/jobs/10/output/all_5_of_28.bnx', 80.59770945960155], ['/data1/bionano-old/compute/jobs/10/output/all_7_of_28.bnx', 80.88649508824305], ['/data1/bionano-old/compute/jobs/10/output/all_22_of_28.bnx', 77.440980368632], ['/data1/bionano-old/compute/jobs/10/output/all_21_of_28.bnx', 77.442724292099228], ['/data1/bionano-old/compute/jobs/10/output/all_18_of_28.bnx', 78.72595426912645], ['/data1/bionano-old/compute/jobs/10/output/all_11_of_28.bnx', 80.91214245146091], ['/data1/bionano-old/compute/jobs/10/output/all_16_of_28.bnx', 78.86442899514176], ['/data1/bionano-old/compute/jobs/10/output/all_6_of_28.bnx', 80.92202843980788], ['/data1/bionano-old/compute/jobs/10/output/all_28_of_28.bnx', 72.64819895477401], ['/data1/bionano-old/compute/jobs/10/output/all_20_of_28.bnx', 77.99202360167413], ['/data1/bionano-old/compute/jobs/10/output/all_13_of_28.bnx', 80.09305622751025]]
```

Obrázek 4.7: Paralelní spuštění skriptu na 20 procesorových jádrech

Výsledky v tabulce 4.2 ukazují vliv paralelizace na rychlost načítání souborů pomocí funkce `load()` popsané dříve v sekci 4.2.2.2. Na lokálním PC, které disponovalo maximálně 12 jádry, se při využití jediného jádra rychlost pohybovala kolem 16–18 MB/s. Při zapojení všech dostupných 12 jader došlo k výraznému zrychlení až na přibližně 42 MB/s. Na serveru byly provedeny testy až do 64 jader. V této konfiguraci byla dosažena rychlost načítání přes 220 MB/s. Tyto výsledky potvrzují, že paralelizace je zvláště efektivní při práci s velkými objemy dat, kde dokáže zásadně zkrátit dobu načítání souborů a výrazně zvýšit celkovou propustnost systému.

Tabulka 4.2: Porovnání rychlosti načítání souborů při různých konfiguracích

Prostředí	Počet jader	Počet souborů	Velikost [MB]	Čas [s]	Rychlost [MB/s]
Lokální PC	1	1	1 635	91,52	17,86
Lokální PC	1	3	2 237	133,32	16,78
Lokální PC	1	3	4 904	277,20	17,69
Lokální PC	3	3	4 904	170,90	28,69
Lokální PC	12	12	4 198	98,96	42,42
Server	1	10	16 720	1241,94	13,46
Server	16	32	52 456	867,51	60,47
Server	32	32	52 456	387,74	135,29
Server	64	64	122 121	501,32	223,60

4.4.3 Přístup k datům

Tabulka 4.3 a graf na obrázku 4.8 znázorňují srovnání časů potřebných ke zpracování souborů různých velikostí při použití funkce `load()` a bez jejího použití. V tomto případě zpracováním rozumíme průchod daty, například výpočet průměrné hodnoty molekul uložených v souboru.

Bez funkce `load()` je nutné soubor pokaždé znovu načítat z disku, což je časově náročné, zejména u větších souborů. Oproti tomu při použití funkce `load()` (popsána v kapitole 4.2.2.2) jsou data po prvním načtení uložena v hashovací tabulce v operační paměti. Sice může být toto počáteční načtení o něco pomalejší, při opakovaném přístupu k datům a jejich následné manipulaci je však tento přístup výrazně výhodnější.

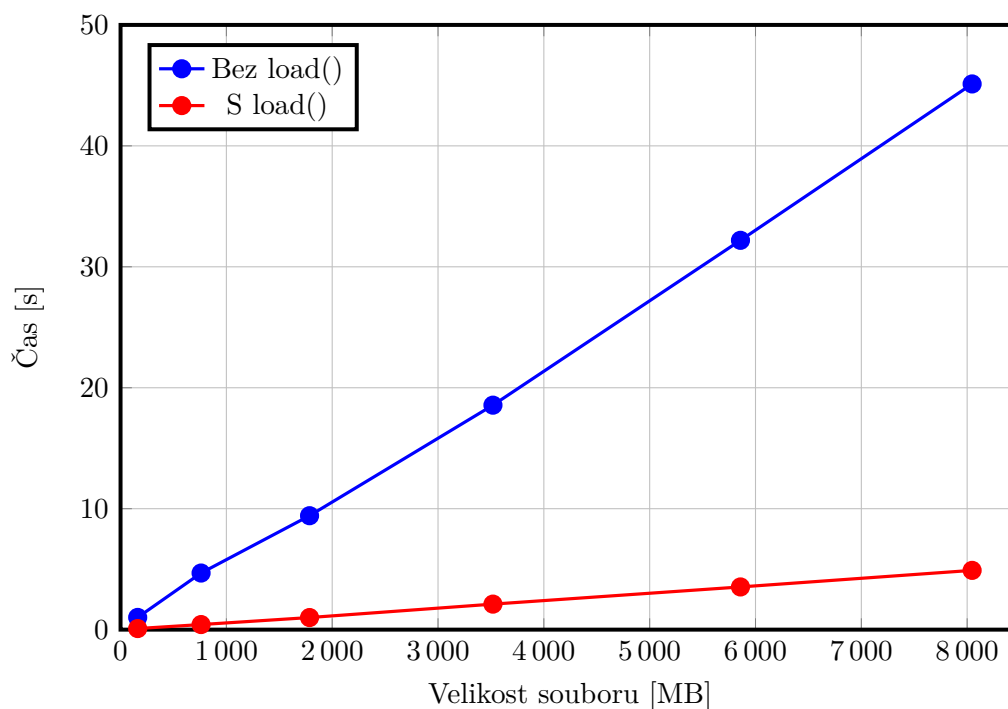
Díky tomu, že jsou data uložena v hashovací tabulce, je možné efektivně přeskočit nerelevantní molekuly a zároveň se na každou molekulu odkazovat přímo, a to včetně všech jejích atributů. Není tak nutné prohledávat celý soubor nebo sekvenčně procházet jednotlivé záznamy, což výrazně urychluje nejen samotné výpočty, ale i další operace, jako je filtrování, statistická analýza nebo vizualizace.

Z výsledků je patrné, že použití funkce `load()` významně zkracuje čas následného zpracování, a rozdíl se s rostoucí velikostí souboru dále zvětšuje. Například u souboru o velikosti 8 GB klesl

čas z přibližně 45 sekund na méně než 5 sekund. Tato optimalizace je obzvlášť výhodná při práci s rozsáhlými datovými soubory, protože může výrazně zlepšit efektivitu.

Tabulka 4.3: Časy zpracování podle velikosti souboru

Velikost souboru [MB]	Čas bez load() [s]	Čas s load() [s]
162	1,01	0,09
761	4,68	0,42
1 786	9,42	1,00
3 519	18,56	2,11
5 858	32,19	3,53
8 047	45,12	4,90



Obrázek 4.8: Porovnání času zpracování v závislosti na velikosti souboru

4.4.4 Výhoda použití serverového prostředí

Testování na serverovém prostředí ukázalo výrazné výhody díky využití velké operační paměti o kapacitě 7 TB. Při převodu a zpracování dat se veškeré informace ukládají přímo do RAM, což

umožňuje rychlé indexování a přímý přístup k jednotlivým záznamům bez nutnosti opakovaného načítání z pomalejších úložných médií. Díky tomu bylo možné efektivně načíst a zpracovat výrazně větší množství souborů, které by na běžných pracovních počítačích s omezenou pamětí nebylo možné vůbec otevřít. Další výhodou je možnost současného využití více procesorových jader, což výrazně urychlilo paralelní zpracování dat a provádění náročnějších výpočtů. Kombinace dostatečné paměti a paralelizace přispěla k celkovému zrychlení práce se soubory Bionano.

Kapitola 5

Závěr

Vývoj nástroje pro zpracování dat optického mapování pro mě představoval velkou a zajímavou výzvu, která spojovala technické aspekty programování s náročností a specifickostí biomedicínských dat. Během práce jsem se musel nejen seznámit se specifickými datovými formáty BNX, CMAP a XMAP, ale také pochopit jejich hlubší význam a úlohu v biologických analýzách. Nástroj jsem vyvíjel flexibilně a s využitím iterativního přístupu, který mi umožňoval nejen průběžně reagovat na nové požadavky a poznatky, ale také postupně ladit a optimalizovat jeho funkčnost podle aktuálních potřeb. Tento přístup mi poskytl dostatečný prostor pro experimentování s různými variantami a rychlé zavádění vylepšení, což mi umožnilo ověřit různé metody a sledovat jejich praktické výsledky. Bylo pro mě také zajímavé navrhnout některé řešení, například paralelní spouštění jednotlivých částí zpracování, a sledovat výsledné zrychlení a efektivitu celého procesu. Zpočátku bylo největší výzvou pochopit, jakým způsobem je potřeba s Bionano soubory zacházet a jaké konkrétní funkce a výstupy se od vyvíjeného nástroje očekávají. Postupně jsem však začal vnímat, jak informatické nástroje a algoritmy mohou zásadním způsobem usnadnit práci, zrychlit analýzy a přispět k rychlejšímu dosažení vědeckých objevů a poznatků. Celý projekt mi ukázal, jak důležité je navrhovat technická řešení s ohledem na konkrétní potřeby, jejich praktické využití a také budoucí rozšiřování. I když vytvořený nástroj má samozřejmě své limity, věřím, že názorně demonstruje potenciál spojení informatiky s moderní biologii a otevírá prostor pro další zlepšení a rozšíření podobných přístupů v budoucnu.

Literatura

1. AMAZON WEB SERVICES. *The Difference Between Hadoop and Spark* [online]. 2025. [cit. 2025-03-15]. Dostupné z: <https://aws.amazon.com/compare/the-difference-between-hadoop-vs-spark>.
2. *Apache Hadoop* [online]. Databricks, 2025. [cit. 2025-01-06]. Dostupné z: <https://www.databricks.com/glossary/hadoop>.
3. BIONANO GENOMICS. *Bionano Genomics* [online]. 2025. [cit. 2024-11-18]. Dostupné z: <https://bionano.com/>.
4. DI TOMMASO, Paolo; CHATZOU, Maria; FLODEN, Efren; BARJA, Pere; PALUMBO, Emilio; NOTREDAME, Cedric. Nextflow enables reproducible computational workflows. *Nature Biotechnology*. 2017, roč. 35, č. 4, s. 316–319. Dostupné z DOI: <https://doi.org/10.1038/nbt.3820>.
5. DREMSEK, Paul; SCHWARZ, Thomas; WEIL, Beatrix; MALASHKA, Alina; LACCONE, Franco; NEESEN, Jürgen. Optical Genome Mapping in Routine Human Genetic Diagnostics—Its Advantages and Limitations. *Genes* [online]. 2021, roč. 12, č. 12 [cit. 2025-04-05]. ISSN 2073-4425. Dostupné z DOI: <https://doi.org/10.3390/genes12121958>.
6. ECMA INTERNATIONAL. *The JSON Data Interchange Syntax* [online]. 2017. [cit. 2025-04-10]. Dostupné z: <https://www.ecma-international.org/publications-and-standards/standards/ecma-404/>.
7. GENOMICS, Bionano. *CMAF File Format Specification Sheet* [online]. 2017. [cit. 2025-04-07]. Dostupné z: <https://bionanogenomics.com/wp-content/uploads/2017/03/30039-CMAF-File-Format-Specification-Sheet.pdf>.
8. GENOMICS, Bionano. *XMAP File Format Specification Sheet* [online]. 2017. [cit. 2025-04-04]. Dostupné z: <https://bionanogenomics.com/wp-content/uploads/2017/03/30040-XMAP-File-Format-Specification-Sheet.pdf>.
9. GENOMICS, Bionano. *BNX File Format Specification Sheet* [online]. 2020. [cit. 2025-04-05]. Dostupné z: <https://bionanogenomics.com/wp-content/uploads/2018/04/30038-BNX-File-Format-Specification-Sheet.pdf>.

10. HANDZUŠOVÁ, Kristína. *Celogenomové optické mapování – nová éra cytogenetiky?* [online]. Brno, 2021 [cit. 2025-04-10]. Dostupné z: <https://is.muni.cz/th/h4fi9/>. Bakalářská práce. Masarykova univerzita, Přírodovědecká fakulta.
11. NATIONAL CENTER FOR BIOTECHNOLOGY INFORMATION. *Bionano Whole Genome Map Files* [online]. 2024. [cit. 2025-04-08]. Dostupné z: <https://www.ncbi.nlm.nih.gov/genbank/bionanomaps/>.
12. NEXTFLOW TEAM. *Nextflow Documentation* [online]. 2024. [cit. 2025-04-08]. Dostupné z: <https://www.nextflow.io/docs/latest/index.html>.
13. PYTHON SOFTWARE FOUNDATION. *The Python Language Reference* [online]. 2023. [cit. 2025-04-10]. Dostupné z: <https://docs.python.org/3/reference/>.
14. PYTHON SOFTWARE FOUNDATION. *filecmp — File and Directory Comparisons* [online]. 2025. [cit. 2025-04-06]. Dostupné z: <https://docs.python.org/3/library/filecmp.html>.
15. TOEGELOVÁ, Helena. Optické mapování aneb Jak vidět sekvenci DNA na vlastní oči. *Živa* [online]. 2016 [cit. 2025-01-06]. Dostupné z: <https://ziva.avcr.cz/files/ziva/pdf/opticke-mapovani-aneb-jak-videt-sekvenci-dna-na-vl.pdf>.
16. *What Is Apache Spark?* [online]. Databricks, 2025 [cit. 2025-01-02]. Dostupné z: <https://www.databricks.com/glossary/what-is-apache-spark>.